

RED DRIVER SAFETY SYSTEM FOR FATIGUE AND DISTRACTION MONITORING

A PROJECT REPORT

Submitted by

ABISHEK S **813822244002**

AKSHAYAGIRI K **813822244003**

KARTHIK P **813822244024**

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND BUSINESS SYSTEMS



SARANATHAN COLLEGE OF ENGINEERING

(An Autonomous Institution)

(Affiliated to Anna University, Chennai - 600 025)



ANNA UNIVERSITY: CHENNAI 600 025

APRIL 2026

SARANATHAN COLLEGE OF ENGINEERING

(An Autonomous Institution)
(Affiliated to Anna University, Chennai – 600 025)

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “**AI-POWERED DRIVER SAFETY SYSTEM FOR FATIGUE AND DISTRACTION MONITORING**” is the bonafide work of

ABISHEK S	813822244002
AKSHAYAGIRI K	813822244003
KARTHIK P	813822244024

who carried out the project work under my supervision.

SIGNATURE

Dr. S. VENKATASUBRAMANIAN
M.E., Ph.D.,

HEAD OF THE DEPARTMENT

ASSOCIATE PROFESSOR

**Computer Science and Business
Systems**

**Saranathan College of Engineering,
Panjappur,**

Tiruchirappalli, Tamilnadu-620 012

SIGNATURE

Mr. R. Kannan M.E., (Ph.D).,

SUPERVISOR

ASSISTANT PROFESSOR

**Computer Science and Business
Systems**

**Saranathan College of Engineering,
Panjappur**

Tiruchirappalli, Tamilnadu-620 012

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We sincerely thank **Shri. S. RAVINDRAN**, Secretary, Saranathan College of Engineering, for giving us a platform to realize our project.

We express our sincere thanks to **Dr. D. VALAVAN, M.Tech., Ph.D.**, Principal, Saranathan College of Engineering, for giving us an opportunity and immense cooperation for the successful completion of the project.

We are obliged to **Dr. S. VENKATASUBRAMANIAN, M.E., Ph.D.**, Associate Professor, Head of the Department of Computer Science and Business Systems, Saranathan College of Engineering, for his valuable suggestions and encouragement to our project.

We express our deep gratitude to our project coordinator, **Mr. S. SENTHIL, M.E., (Ph.D.)**, Assistant Professor, Department of Computer Science and Business Systems, Saranathan College of Engineering, who rendered full support during the period of our project work.

We also express our heartfelt thanks to our project supervisor and guide, **Mr. R. KANNAN M.E., (Ph.D.)**, Assistant Professor, Department of Computer Science and Business Systems, Saranathan College of Engineering, for his valuable guidance, ideas, and suggestions.

We would like to thank all our faculty and technical assistants for their full support and help rendered by them in the completion of this project. We are ever thankful to our parents for their moral support.

Above all, we thank God Almighty for His bountiful blessing.

ABSTRACT

Road safety has become a critical concern worldwide due to the increasing number of accidents caused by driver fatigue and distraction. Drowsiness significantly affects a driver's alertness, reaction time, and decision-making ability, leading to a higher risk of accidents. Prolonged driving and lack of rest further increase fatigue, making it essential to develop an effective monitoring system. This project presents a real-time driver drowsiness detection system using computer vision and deep learning to monitor driver behavior and identify early signs of fatigue. The system captures live video using a camera and processes frames using OpenCV and MediaPipe for face detection and feature extraction. Key facial features such as eyes and mouth are analyzed to detect fatigue indicators. Convolutional Neural Network (CNN) models classify eye state and detect yawning with high accuracy. The system also performs temporal analysis by monitoring eye closure duration and yawning frequency to improve detection reliability. Based on these features, the system classifies the driver's condition as alert or drowsy. When fatigue is detected, alerts are generated through audio or visual notifications. The system operates in real time without requiring wearable sensors, making it non-intrusive and suitable for practical use. Experimental results show high accuracy, making the system an efficient and cost-effective solution for improving road safety.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF FIGURES	viii
	LIST OF TABLES	ix
	LIST OF ABBREVIATIONS	x
1	INTRODUCTION	1
	1.1 Problem Statement	2
	1.2 Scope of the Project	2
	1.3 Novelty of the Project	3
2	LITERATURE REVIEW	4
	2.1 Driver Fatigue Detection Based on Heart Rate Variability (HRV)	4
	2.2 Real-Time Driver Fatigue Detection Based on Steering Wheel Movements	5
	2.3 Optimized Driver Safety through Driver Fatigue Detection Methods	5
	2.4 Driver Drowsiness Detection Using Vehicle Dynamics and Machine Learning	6
	2.5 Real-Time Driver Drowsiness Detection Using Computer Vision and Deep Learning	7
	2.6 Driver Drowsiness Detection Using Steering Behavior and Lane Deviation Analysis	8
	2.7 Research Gap Identified	9
3	SYSTEM DESIGN	11
	3.1 System Architecture	11

3.1.1	Input Layer	12
3.1.2	Processing Layer	12
3.1.3	AI Detection Layer	12
3.1.4	Decision Layer	13
3.1.5	Output Layer	13
3.2	Data Flow Overview	13
3.2.1	Input Live Video Feed	14
3.2.2	Face Detection	14
3.2.3	Feature Extraction	14
3.2.4	Parallel Detection Modules	15
3.2.5	Decision Logic	15
3.2.6	Alert Generation	16
3.2.7	Dashboard Display	16
3.2.8	Continuous Monitoring Loop	16
3.3	UML Diagram	16
3.3.1	Use Case Diagram	16
3.3.2	Class Diagram	17
3.3.3	Sequence Diagram	18
3.3.4	Package Diagram	19
3.3.5	Activity Diagram	20
4	SYSTEM ANALYSIS	22
4.1	Existing System	22
4.2	Proposed System	23
4.3	Feasibility Study	24
5	SYSTEM SPECIFICATION	25
5.1	Software Description	25
5.1.1	Frontend Technologies	25
5.1.2	Backend Technologies	25

5.2	Hardware Description	28
5.3	Dataset Description	30
6	SYSTEM IMPLEMENTATION	31
6.1	List Of Modules	31
6.1.1	Video Acquisition & Preprocessing	31
6.1.2	Face Detection & ROI Extraction	32
6.1.3	Eye & Mouth Region Detection	33
6.1.4	CNN-Based Eye State Classification	34
6.1.5	CNN-Based Yawning Detection	34
6.1.6	Temporal Fatigue Analysis	35
6.1.7	Mobile Phone Distraction Detection	36
6.1.8	Feature Fusion & Driver State Classification	36
6.1.9	Alert System	37
6.1.10	Dashboard & Visualization	37
6.1.11	System Integration & Optimization	38
6.1.12	Testing, Evaluation & Documentation	39
7	TESTING	40
7.1	Testing	40
7.1.1	Test Cases	40
8	RESULTS AND PERFORMANCE COMPARISON	42
8.1	Detection Efficiency	42
8.2	Scalability Evaluation	42
8.3	Performance Metrics	43
8.4	Comparison with Existing System	44
8.5	Operational Efficiency Gains	45
9	CONCLUSION AND FUTURE WORK	46

9.1	Conclusion	46
9.2	Future Work	47
APPENDIX - I: SOURCE CODE		48
APPENDIX - II: SCREENSHOTS		59
APPENDIX - III: PUBLICATION		65
REFERENCES		70

LIST OF FIGURES

FIGURE NO.	NAME OF THE FIGURES	PAGE NO.
3.1	System Architecture	11
3.2	Data Flow Diagram	13
3.3	Use Case Diagram	17
3.4	Class Diagram	18
3.5	Sequence Diagram	19
3.6	Package Diagram	20
3.7	Activity Diagram	21
6.1	System Flow Diagram	31
6.2	Video Acquisition & Preprocessing	32
6.3	MediaPipe's Facial Landmarks	33
6.4	Eye and Mouth Region Extraction using MediaPipe	33
6.5	Eye State Classification	34
6.6	Mouth State Classification	35
6.7	Mobile Phone Detection	36
6.8	Driver State Classification	37
6.9	Dashboard	38
6.10	Dashboard	38

LIST OF TABLES

TABLE NO.	NAME OF THE TABLES	PAGE NO.
5.1	Software Specifications	28
5.2	Hardware Specifications	29
7.1	Test Cases	40
8.1	Performance Metrics	43
8.2	Comparison with Existing System	44

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
AI	Artificial Intelligence
CNN	Convolutional Neural Network
CV	Computer Vision
ROI	Region of Interest
ML	Machine Learning
DL	Deep Learning
RGB	Red Green Blue (Color Model)
BGR	Blue Green Red (Color Format in OpenCV)
FPS	Frames Per Second
TTS	Text-To-Speech
YOLO	You Only Look Once
API	Application Programming Interface
GUI	Graphical User Interface
IDE	Integrated Development Environment
OS	Operating System
RAM	Random Access Memory
GPU	Graphics Processing Unit
CPU	Central Processing Unit
EAR	Eye Aspect Ratio
MAR	Mouth Aspect Ratio

CHAPTER 1

INTRODUCTION

Road transportation plays a vital role in modern society, enabling the efficient movement of people and goods. However, the rapid increase in the number of vehicles has led to a rise in traffic accidents, many of which are caused by driver fatigue and distraction. Driver drowsiness reduces alertness, slows reaction time, and affects decision-making ability, making it a major cause of road accidents worldwide. In addition, distractions such as mobile phone usage further increase risk by diverting the driver's attention from the road.

Traditional approaches for detecting driver fatigue include monitoring steering behavior, physiological signals, and vehicle dynamics. Methods based on steering patterns are non-intrusive but often detect fatigue only after it affects driving performance. Physiological approaches, such as heart rate variability analysis, provide accurate results but require wearable sensors, making them less practical for real-world use. Recent studies have explored machine learning techniques using vehicle dynamics data; however, their performance is influenced by external conditions such as road type and traffic.

With the advancement of computer vision and deep learning, more reliable and non-intrusive driver monitoring systems have been developed. These systems use facial features such as eye closure and yawning to detect fatigue in real time. Hybrid approaches combining multiple detection methods have also been proposed to improve accuracy. This project proposes an AI-powered driver safety system for fatigue and distraction monitoring using real-time video input. The system uses computer vision and Convolutional Neural Networks (CNNs) to detect eye closure, yawning, and mobile phone usage. By analyzing these indicators, the system identifies the driver's state and generates timely alerts to prevent accidents.

1.1 Problem Statement

Driver fatigue and distraction are major contributors to road accidents, leading to severe injuries, fatalities, and economic losses. Many drivers experience drowsiness due to long driving hours, lack of rest, or monotonous driving conditions. In such situations, the driver's ability to respond to sudden events is significantly reduced. Additionally, distractions such as mobile phone usage further increase the risk by diverting attention from driving tasks.

Existing systems for detecting driver fatigue often rely on physiological sensors or vehicle-based parameters such as steering behavior. These systems have several limitations:

- Physiological methods require wearable sensors, causing discomfort
- Vehicle-based methods detect fatigue only at later stages
- Traditional vision-based systems lack robustness in real conditions
- Poor performance under low lighting and head pose variations
- High implementation cost and complexity

Due to these limitations, there is a need for a **real-time, non-intrusive, and accurate system** that can detect both drowsiness and distraction effectively.

1.2 Scope of the Project

The scope of this project encompasses the design, development, and evaluation of an AI-Powered Driver Safety System capable of:

- A real-time driver drowsiness and distraction detection system using computer vision and deep learning techniques.
- Generate timely alerts (audio or visual) when unsafe driver conditions are detected.

- Identify driver distraction, especially mobile phone usage
- Integrate multiple detection modules for improving accuracy and reducing false alarms.
- Design a system that is cost-effective, non-intrusive, and suitable for real-world implementation.
- Provide a user interface or dashboard for displaying the driver's status in real time.
- Improve road safety through early detection of fatigue

1.3 Novelty of the Project

The proposed driver drowsiness and distraction detection system presents a robust and real-time solution by integrating multiple behavioral indicators using advanced deep learning techniques. Unlike conventional approaches that rely on a single parameter, this system combines eye state analysis, yawning detection, and distraction monitoring to improve detection accuracy and reliability. The use of Convolutional Neural Networks (CNNs) enables automatic feature learning, reducing dependence on manual feature extraction and improving performance under varying environmental conditions. The system is designed to be non-intrusive and efficient, making it suitable for practical implementation in real-world driving scenarios.

- Integration of multiple indicators such as eye closure, yawning, and distraction for improved accuracy
- Non-intrusive design using camera-based monitoring without requiring wearable sensors
- User-friendly dashboard for real-time monitoring and visualization of driver status
- Real-time processing with immediate voice alert generation for timely driver intervention

CHAPTER 2

LITERATURE REVIEW

2.1 Driver Fatigue Detection Based on Heart Rate Variability (HRV)

Authors: J. Vicente, P. Laguna, A. Bartra, R. Bailón

Year: 2016

This study proposes a driver fatigue detection method based on the analysis of heart rate variability (HRV), which reflects the functioning of the autonomic nervous system. The system collects physiological signals from the driver using sensors and analyzes variations in heartbeat intervals to determine fatigue levels. The idea behind this approach is that fatigue affects the balance between sympathetic and parasympathetic nervous activity, which can be captured through HRV analysis. By monitoring these variations, the system can detect early signs of drowsiness before visible behavioral changes occur. One of the major merits of this method is its high accuracy, as it directly measures physiological signals related to the driver's internal state. It is also capable of detecting fatigue at an early stage, making it useful for preventive systems. However, the method has significant limitations. It requires wearable sensors or electrodes, which can cause discomfort for the driver during long journeys. Additionally, the system setup is complex and costly, making it less practical for real-world implementation. Environmental noise and sensor placement can also affect signal quality. Despite these drawbacks, HRV-based approaches provide valuable insights into fatigue detection and contribute to the development of more advanced driver monitoring systems.

2.2 Real-Time Driver Fatigue Detection Based on Steering Wheel Movements

Authors: Luis M. Bergasa, Jesús Nuevo, Miguel A. Sotelo

Year: 2006

This research presents a driver fatigue detection system based on analyzing steering wheel movements during driving. The system monitors variations in steering behavior to identify patterns associated with fatigue. The underlying idea is that a fatigued driver exhibits irregular or delayed steering corrections due to reduced alertness and slower reaction time. By continuously tracking steering angle and movement patterns, the system attempts to detect abnormal driving behavior. One of the main advantages of this approach is that it is non-intrusive and does not require cameras or wearable sensors, making it easy to integrate into existing vehicle systems. It uses data from vehicle sensors, which are already available in many modern vehicles. However, the system has several limitations. Its accuracy depends heavily on external factors such as road conditions, traffic, and driving style. Sudden maneuvers or uneven roads may lead to incorrect detection. Moreover, this method detects fatigue only after it begins to affect driving performance, making it less effective for early warning systems. Despite these limitations, this approach has contributed to the development of vehicle-based monitoring systems and highlights the importance of analyzing driving behavior.

2.3 Optimized Driver Safety through Driver Fatigue Detection Methods

Authors: Omar Wathiq, Bhavna D. Ambudkar

Year: 2017

This study explores various driver fatigue detection techniques and proposes an optimized framework to improve driver safety. The research reviews multiple approaches, including physiological signal analysis, driver behavior monitoring, and vision-based systems. The idea behind this work is to combine the strengths of different detection methods to overcome the limitations of individual approaches. The study emphasizes the importance of integrating multiple indicators such as eye blinking, yawning, and head movement to improve detection accuracy. One of the key merits of this work is its comprehensive analysis of different fatigue detection techniques and its focus on optimization. It highlights the need for combining multiple methods to achieve better performance and reliability. The proposed framework aims to enhance detection accuracy and reduce false alarms by using a hybrid approach. However, the study does not provide a fully implemented system, and practical deployment challenges are not addressed in detail. Additionally, combining multiple methods may increase system complexity and computational requirements. Despite these limitations, the research provides valuable insights into designing more effective and reliable driver monitoring systems.

2.4 Driver Drowsiness Detection Using Vehicle Dynamics and Machine Learning

Authors: M. Khan, S. Rahman, A. Ahmed

Year: 2021

This study presents a driver drowsiness detection system based on vehicle dynamics using machine learning techniques. The approach focuses on analyzing driving behavior parameters such as steering wheel angle, lane position, vehicle speed, and acceleration patterns to detect signs of fatigue. The core idea is that drowsy drivers exhibit noticeable deviations in driving patterns, such as inconsistent steering corrections and lane drifting. The collected vehicle data is

processed and fed into machine learning models to classify the driver's state as alert or drowsy. One of the main advantages of this method is that it is non-intrusive, as it does not require cameras or wearable sensors. It utilizes data from existing vehicle sensors, making it easier to implement in modern vehicles. Additionally, the use of machine learning improves detection accuracy compared to traditional rule-based methods. However, this approach has certain limitations. Its performance is highly dependent on external conditions such as road type, traffic, and weather. Variations in individual driving styles may also affect the model's reliability. Moreover, this method detects fatigue only after it starts influencing driving behavior, limiting its ability to provide early warnings. Despite these challenges, the study demonstrates the potential of combining vehicle dynamics with machine learning for practical fatigue detection systems.

2.5 Real-Time Driver Drowsiness Detection Using Computer Vision and Deep Learning

Authors: R. Sharma, P. Verma

Year: 2022

This research proposes a real-time driver drowsiness detection system using computer vision and deep learning techniques. The system captures video input and analyzes facial features such as eye blinking and mouth movement to detect fatigue. The approach combines image processing with deep learning models to improve detection accuracy. The idea is to create a non-intrusive and efficient system that can operate in real-world conditions. One of the key merits of this method is its ability to perform real-time detection with high accuracy. The use of deep learning improves robustness against variations in lighting and facial appearance. The system is also cost-effective, as it uses standard cameras and software-based processing. However, the method may still face challenges under low-light conditions or when the driver's face is partially occluded. Additionally,

real-time processing requires optimization to ensure smooth performance. Despite these limitations, the system demonstrates the effectiveness of combining computer vision and deep learning for practical driver monitoring applications.

2.6 Driver Drowsiness Detection Using Steering Behavior and Lane Deviation Analysis

Authors: L. Zhang, Y. Chen, H. Li

Year: 2022

This study proposes a driver drowsiness detection system based on analyzing steering behavior and lane deviation patterns using vehicle dynamics data. The main idea of this approach is that driver fatigue directly affects vehicle control, leading to irregular steering movements and unintended lane drifting. The system collects data from onboard vehicle sensors, including steering wheel angle, lateral position, and lane deviation, and analyzes these parameters over time. Machine learning techniques are applied to identify patterns associated with drowsy driving and classify the driver's condition. One of the key advantages of this method is that it is completely non-intrusive, as it does not require cameras or wearable devices. It can be easily integrated into modern vehicles using existing sensor data. Additionally, the use of multiple vehicle parameters improves detection compared to single-parameter systems. However, the method has several limitations. Its accuracy is highly influenced by external factors such as road curvature, traffic conditions, and driving environment. Drivers with different driving styles may produce varying patterns, which can affect model consistency. Moreover, this approach detects fatigue only after it begins to influence driving behavior, making early detection difficult. Despite these challenges, the system demonstrates the effectiveness of vehicle dynamics analysis for practical driver monitoring applications.

From the analysis of existing methods, it is observed that performance-based systems detect fatigue only after it affects driving behavior, while physiological methods are accurate but intrusive. Vision-based systems provide a balance between accuracy and practicality but require improvements in robustness. Therefore, there is a need for a real-time, non-intrusive system that integrates multiple indicators using advanced computer vision and deep learning techniques to achieve reliable driver monitoring.

2.7 Research Gap Identified

Despite significant advancements in driver drowsiness detection, existing systems still face several limitations that affect their performance and real-world applicability. Various approaches such as driver performance monitoring, physiological signal analysis, and vision-based techniques have been explored. However, each method has its own drawbacks, and most systems fail to provide a comprehensive and reliable solution. The complexity of driver fatigue requires a multi-dimensional analysis, which is often missing in traditional approaches.

Identified Gaps in Existing Systems:

- Lack of multi-feature integration for comprehensive analysis
- Intrusive nature of physiological methods requiring wearable sensors
- Late detection in driver performance-based methods, as fatigue is identified only after affecting driving behavior
- Sensitivity of traditional vision-based systems to lighting conditions, head pose variations, and facial occlusions
- Poor performance under low-light or night-time driving conditions
- Absence of distraction detection, especially mobile phone usage
- Reduced robustness and reliability in diverse real-world environments

To overcome these limitations, there is a need for an advanced driver monitoring system that combines multiple detection techniques in a non-intrusive

manner. The integration of computer vision and deep learning enables accurate feature extraction and real-time analysis, improving detection performance. A system that incorporates multiple fatigue indicators along with distraction detection can provide a more reliable and practical solution. Therefore, the proposed system aims to address these gaps by developing a real-time, efficient, and scalable driver drowsiness detection framework suitable for real-world applications.

CHAPTER 3

SYSTEM DESIGN

3.1 System Architecture

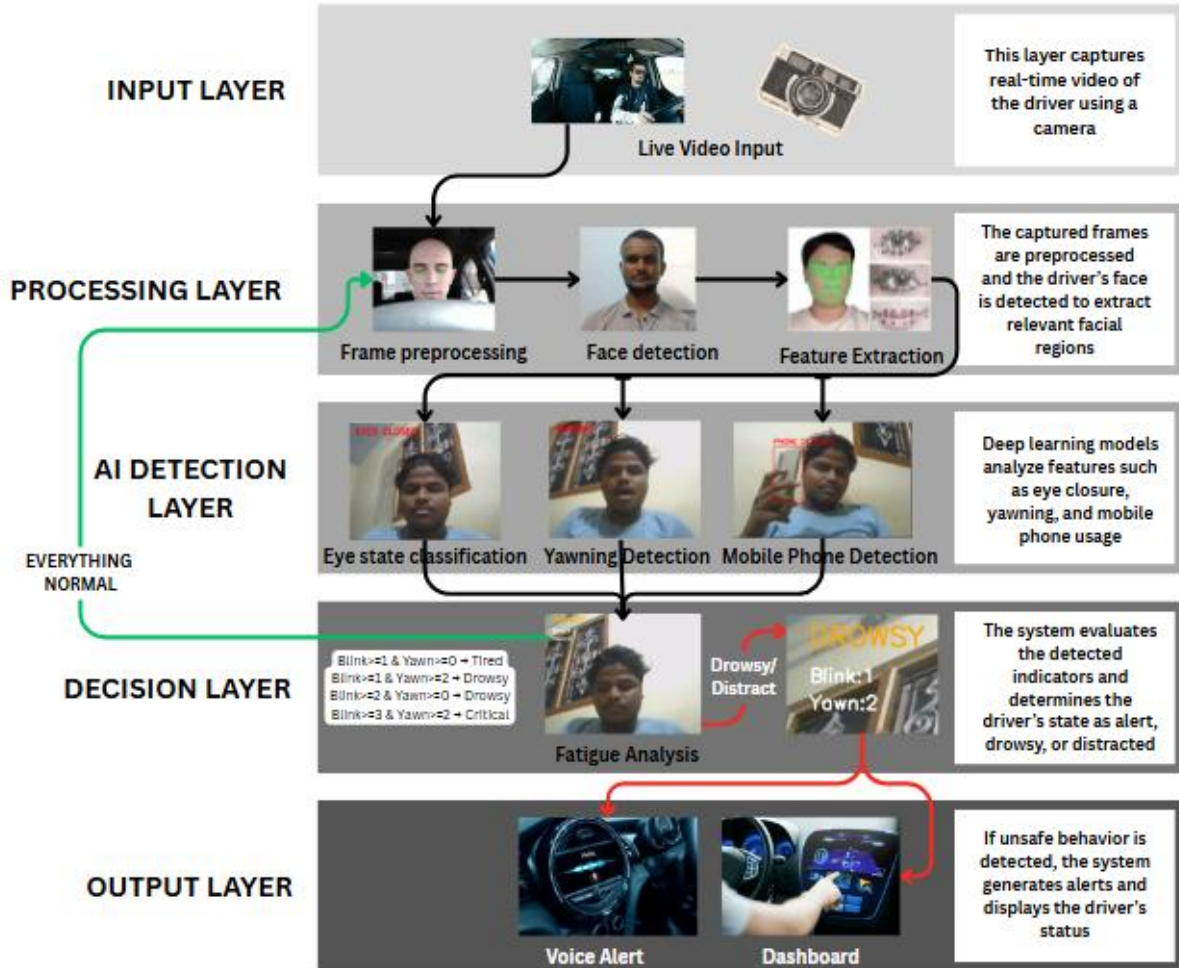


Fig 3.1 System Architecture

Fig.3.1 shows a layered architecture designed to process real-time video data and analyze driver behavior efficiently. The system captures live video input from a camera and processes it through several stages including preprocessing, feature extraction, artificial intelligence-based detection, and decision analysis. Each layer performs a specific function that contributes to identifying signs of driver fatigue or distraction. The architecture combines computer vision techniques and deep learning models to analyze facial features such as eye

closure, yawning, and mobile phone usage. Based on the detected indicators, the system evaluates the driver's condition and generates alerts when unsafe behavior is detected. This structured architecture ensures reliable monitoring, real-time performance, and improved road safety.

3.1.1 Input Layer

The input layer is responsible for capturing real-time video of the driver using a camera placed inside the vehicle cabin. The camera continuously records the driver's facial region while driving. This live video stream serves as the primary source of data for the system. The captured frames are forwarded to the next stage for further processing and analysis.

3.1.2 Processing Layer

The processing layer prepares the captured video frames for analysis. In this stage, frame preprocessing operations such as resizing and normalization are performed to improve image quality. The system then detects the driver's face from each frame and isolates important facial regions. After face detection, feature extraction techniques are used to identify key components such as the eyes and mouth, which are essential for detecting fatigue-related behavior.

3.1.3 AI Detection Layer

In the AI detection layer, deep learning models analyze the extracted facial features to detect signs of drowsiness and distraction. The eye state classification model determines whether the driver's eyes are open or closed. The yawning detection module identifies mouth movements associated with fatigue, while the mobile phone detection module detects potential driver distraction caused by phone usage. These AI models help identify behavioral patterns that indicate reduced driver alertness.

3.1.4 Decision Layer

The decision layer evaluates the outputs generated by the AI detection modules. The system analyzes indicators such as blink frequency, eye closure duration, and yawning patterns to determine the driver's condition. Based on predefined thresholds and fatigue analysis, the driver's state is classified into categories such as alert, tired, drowsy, or critical. This layer plays an important role in determining whether the driver requires a warning.

3.1.5 Output Layer

The output layer is responsible for notifying the driver and displaying the monitoring results. If the system detects unsafe driver behavior, it generates alerts such as voice warnings to regain the driver's attention. In addition, a dashboard interface displays the driver's current state and system status in real time. This layer ensures that the driver receives immediate feedback to prevent potential accidents.

3.2 Data Flow Overview

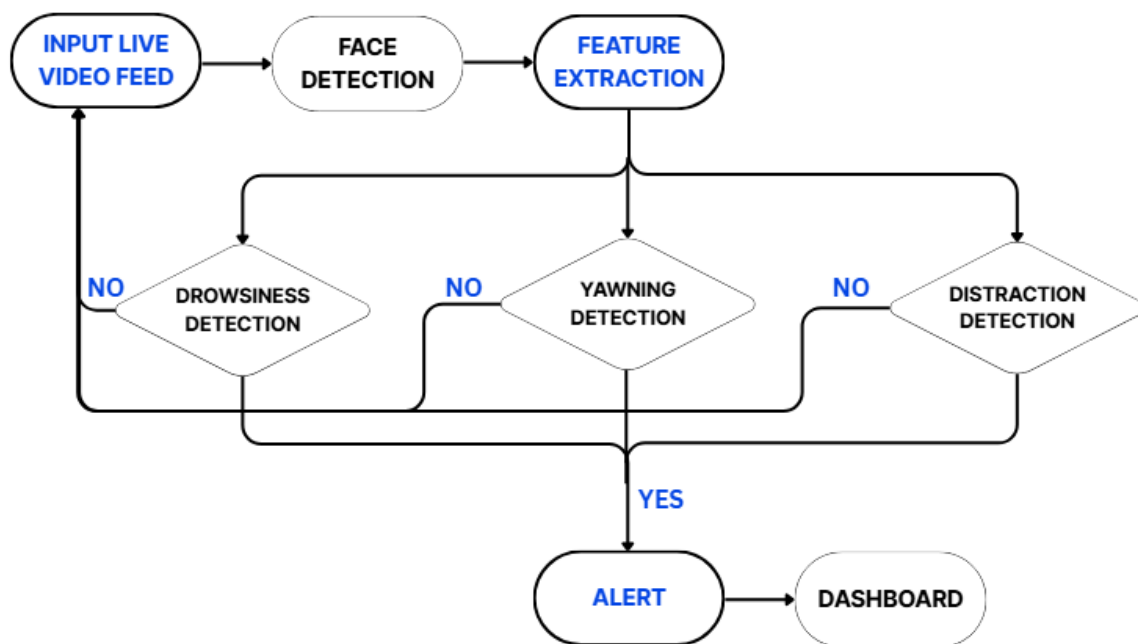


Fig 3.2 Data Flow Diagram

The data flow diagram (Fig.3.2) represents the complete working process of the driver drowsiness detection system, illustrating how data is captured, processed, analyzed, and converted into meaningful output. The system operates in a continuous loop to ensure real-time monitoring of the driver's condition.

3.2.1 Input Live Video Feed

The system begins by capturing a live video stream using a camera positioned inside the vehicle. This camera continuously records the driver's facial region during driving. The video is divided into individual frames, which are processed sequentially. This continuous input ensures that the system always has up-to-date information about the driver's behavior. The quality of this input directly affects the accuracy of the entire system.

3.2.2 Face Detection

Once the video frames are captured, each frame is passed to the face detection module. This module identifies the presence and position of the driver's face within the frame. By isolating the face, the system removes unnecessary background information, reducing computational complexity. Accurate face detection is essential because all further analysis depends on correctly identifying the facial region.

3.2.3 Feature Extraction

After detecting the face, the system extracts key facial features such as the eyes, mouth, and sometimes head orientation. These features are critical for detecting signs of fatigue and distraction. The system focuses on these regions to reduce processing time and improve detection accuracy. Feature extraction acts as a bridge between raw image data and intelligent analysis.

3.2.4 Parallel Detection Modules

Once features are extracted, the system processes them through three parallel detection modules. These modules operate simultaneously to analyze different aspects of driver behavior.

a) Drowsiness Detection (Eye Analysis)

This module evaluates the eye region to determine whether the eyes are open or closed. It tracks eye closure duration and blinking patterns. If the eyes remain closed for an extended period, it indicates drowsiness. If no abnormal eye behavior is detected, the system continues monitoring without triggering alerts.

b) Yawning Detection (Mouth Analysis)

This module analyzes the mouth region to detect yawning. Frequent or prolonged yawning is considered a strong indicator of fatigue. The system observes the opening and closing patterns of the mouth to identify yawning behavior. If yawning is not detected, the system moves forward without interruption.

c) Distraction Detection

This module detects driver distraction, particularly mobile phone usage. It checks whether the driver is looking away from the road or interacting with a device. Distraction detection ensures that the system not only monitors fatigue but also unsafe driving behavior caused by lack of attention.

3.2.5 Decision Logic

The outputs from all detection modules are evaluated together in the decision stage. The system checks whether any unsafe condition is present. If all modules return a NO condition, the system continues monitoring in a loop. However, if any module detects abnormal behavior (YES condition), the system identifies it as a risk state.

3.2.6 Alert Generation

When a risk is detected, the system immediately generates an alert. The alert may be in the form of a sound, warning message, or visual signal. This real-time notification helps the driver become aware of their condition and take corrective action such as stopping the vehicle or regaining focus.

3.2.7 Dashboard Display

The final output is displayed on a dashboard interface. The dashboard shows the driver's current state, such as alert, drowsy, or distracted. It also provides continuous updates and monitoring results. This helps in tracking driver behavior over time and improves system transparency.

3.2.8 Continuous Monitoring Loop

One of the key features of the system is its continuous operation. After completing one cycle, the system returns to the input stage and processes the next frame. This loop ensures uninterrupted monitoring and immediate detection of any unsafe behavior.

3.3 UML Diagrams

3.3.1 Use Case Diagram

The use case diagram illustrates the interaction between the Driver (actor) and the Driver Drowsiness Detection System. The driver initiates the system, which performs various functions such as capturing video, detecting the face, extracting features, detecting eye state, and identifying yawning.

The system then analyzes fatigue and generates alerts if necessary. The results are displayed on the dashboard for monitoring. This diagram highlights the functional requirements of the system and shows how the user interacts with different system operations. It provides a clear understanding of system behavior from the user's perspective.

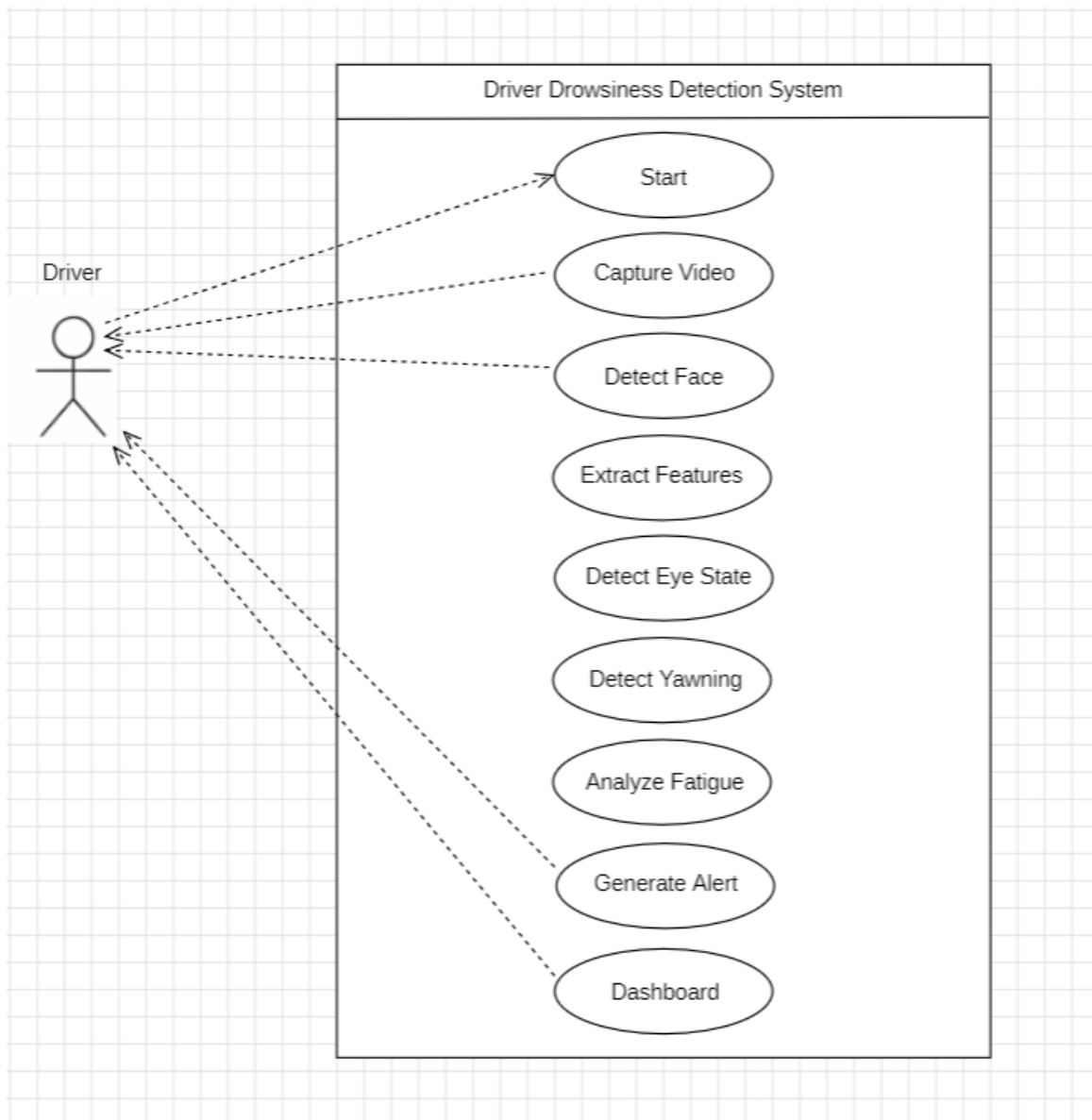


Fig 3.3 Use Case Diagram

3.3.2 Class Diagram

The class diagram represents the structural design of the driver drowsiness detection system by illustrating the different classes, their attributes, and the relationships between them. The system begins with the VideoCapture class, which is responsible for acquiring real-time video frames. These frames are passed to the FaceDetector class, which identifies the driver's face. The detected face is then processed by the FeatureExtractor, which extracts important regions such as the eyes and mouth.

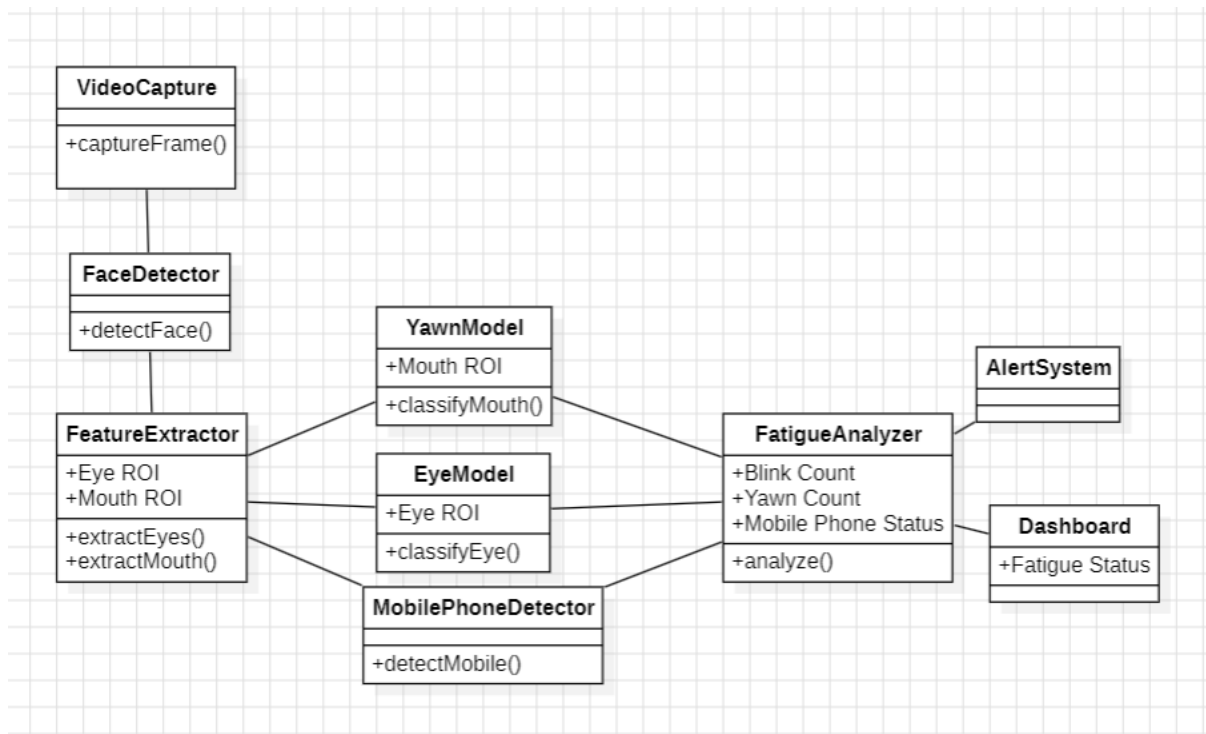


Fig 3.4 Class Diagram

The extracted features are analyzed using deep learning models, namely the EyeModel for eye state classification and the YawnModel for yawning detection. Additionally, the MobilePhoneDetector identifies distraction caused by phone usage. The outputs from these models are combined in the FatigueAnalyzer, which evaluates parameters like blink count, yawning frequency, and distraction status to determine the driver's condition. Finally, the AlertSystem generates warnings, and the Dashboard displays the driver's status. This diagram clearly shows how different components interact to form a complete system.

3.3.3 Sequence Diagram

The sequence diagram illustrates the step-by-step interaction between different components over time. The process begins when the Driver starts the system. The system sends a request to the Camera to capture video frames. Once the frames are received, preprocessing is performed to prepare the data for analysis.

The processed frames are then sent to the Models, where eye state and yawning detection are carried out. Based on these predictions, the system determines the driver's condition. If unsafe behavior is detected, an alert message is generated and sent to the Alert module. At the same time, the processed information and analytics are displayed on the Dashboard. This diagram highlights the real-time data flow and interaction between system components, ensuring continuous monitoring and immediate response.

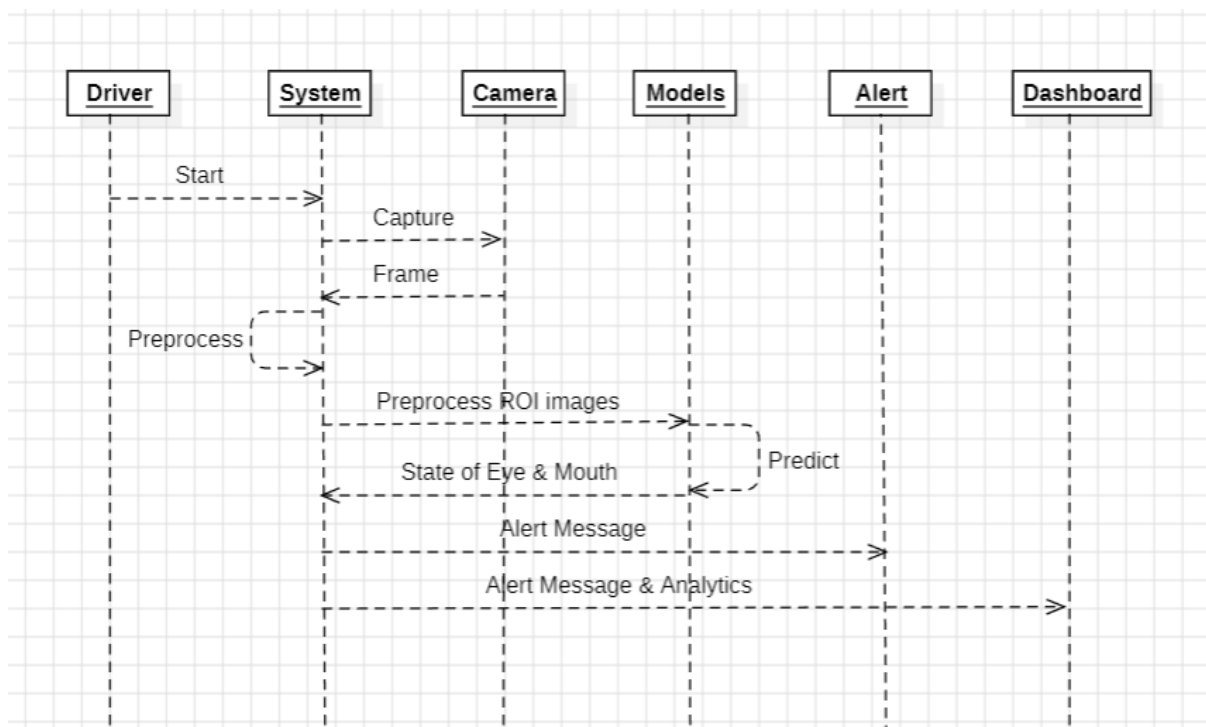


Fig 3.5 Sequence Diagram

3.3.4 Package Diagram

The package diagram provides a high-level view of how the system is organized into different functional modules. The system is divided into five main packages: Input Module, Processing Module, Detection Module, Fatigue Analysis Module, and Output Module.

The Input Module is responsible for capturing real-time video data. The Processing Module performs tasks such as face detection and feature extraction. The Detection Module includes deep learning models that identify eye closure

and yawning. The Fatigue Analysis Module evaluates these indicators to determine the driver's condition. Finally, the Output Module generates alerts and displays results through the dashboard.

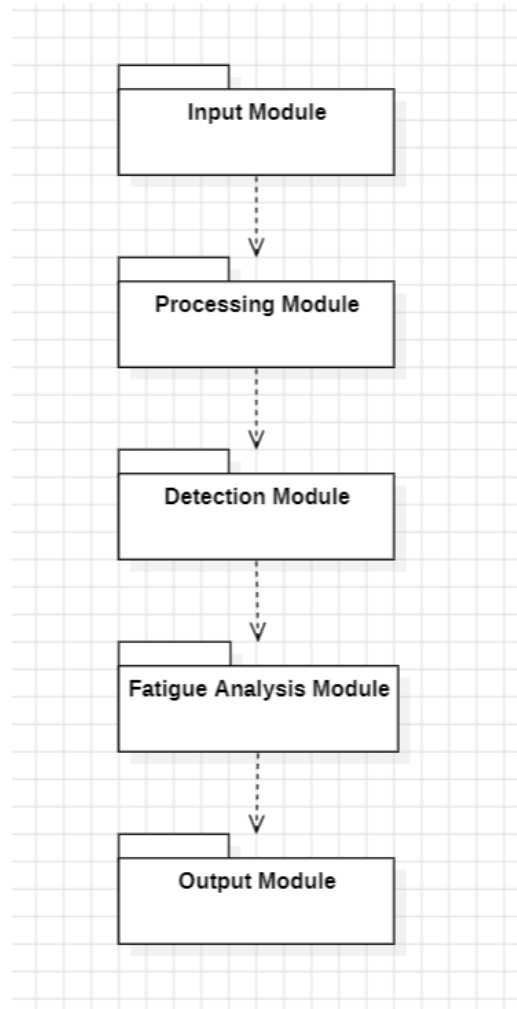


Fig 3.6 Package Diagram

This modular structure improves system organization, scalability, and maintainability, allowing easy integration of additional features in the future.

3.3.5 Activity Diagram

The activity diagram represents the workflow of the system from start to end. The process begins with system initialization and video capture. The system then detects the driver's face and extracts relevant features such as eyes and mouth.

Next, the system performs eye detection and yawning detection to identify fatigue indicators. These indicators are analyzed to determine the driver's condition. A decision is made based on whether the driver is drowsy or not. If drowsiness is detected, an alert is generated; otherwise, the system continues monitoring. Finally, the results are displayed, and the process repeats continuously.

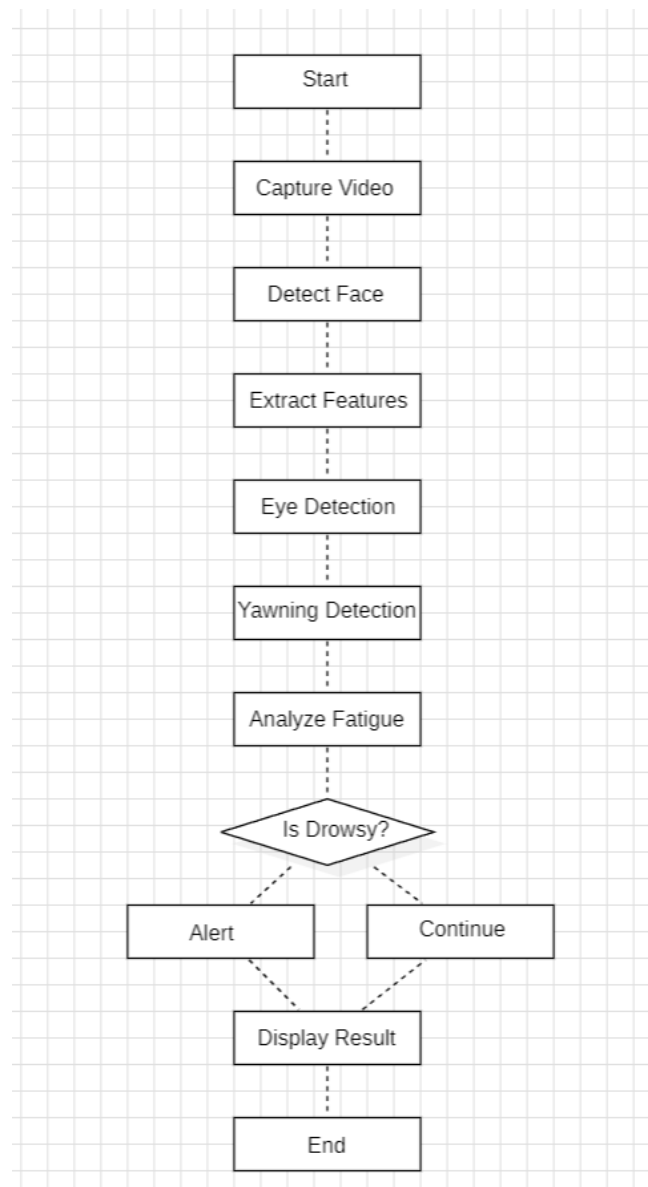


Fig 3.7 Activity Diagram

This diagram clearly explains the logical flow of operations and decision-making process within the system.

CHAPTER 4

SYSTEM ANALYSIS

System analysis involves studying the existing driver monitoring methods, identifying their limitations, and defining the proposed system that overcomes these drawbacks. This chapter explains the currently used drowsiness detection approaches and the improvements introduced in the proposed system. A feasibility study is also conducted to ensure that the system can be practically implemented in real-world scenarios.

4.1 Existing System

Existing driver drowsiness detection systems mainly rely on traditional approaches such as driver performance monitoring, physiological signal analysis, and basic vision-based techniques. These systems are designed to detect fatigue based on limited indicators and often lack real-time efficiency and robustness.

Major limitations of the existing system include:

- Dependence on single-parameter detection (eye blink or steering behavior)
- Requirement of wearable sensors in physiological methods
- Poor performance under varying lighting conditions
- Inability to detect early-stage fatigue
- Lack of distraction detection such as mobile phone usage
- Limited real-time processing capability

Driver performance-based systems analyze vehicle behavior such as lane deviation and steering patterns. However, these indicators appear only after fatigue has already affected driving. Physiological systems provide accurate results but require sensors attached to the driver, causing discomfort.

Traditional vision-based systems rely on handcrafted features and simple image processing techniques. These methods are sensitive to lighting conditions, head movements, and occlusions, which reduces their effectiveness in real-world environments. As a result, existing systems often fail to provide reliable and timely detection.

4.2 Proposed System

The proposed system is an AI-based driver drowsiness and distraction detection system designed to improve accuracy, reliability, and real-time performance.

Key improvements in the proposed system include:

- Use of CNN models for accurate feature extraction and classification
- Integration of multiple fatigue indicators such as eye closure and yawning
- Inclusion of mobile phone detection for distraction analysis
- Real-time video processing using computer vision techniques
- Non-intrusive monitoring using a camera
- Alert system for immediate driver notification

The system continuously captures video input from the driver and processes it using OpenCV and MediaPipe for face and feature detection. Deep learning models analyze the extracted features to identify signs of fatigue and distraction. Based on the analysis, the system determines the driver's state and generates alerts when unsafe behavior is detected.

Additionally, a dashboard is used to display real-time status and monitoring results. This integrated approach improves detection accuracy and ensures timely intervention, thereby enhancing road safety.

4.3 Feasibility Study

Technical Feasibility:

The system uses widely available technologies such as Python, OpenCV, TensorFlow, and MediaPipe. These tools support real-time video processing and deep learning model implementation, making the system technically feasible and reliable.

Economic Feasibility:

The proposed system is cost-effective as it uses open-source software and a standard webcam for input. There is no need for expensive sensors or specialized hardware, making it affordable for real-world deployment.

Operational Feasibility:

The system is user-friendly and operates automatically without requiring manual input. It provides real-time alerts and visual feedback, making it easy to use in practical driving conditions. The non-intrusive design ensures driver comfort and acceptance.

CHAPTER 5

SYSTEM SPECIFICATION

5.1 Software Description

5.1.1 Frontend Technologies

Streamlit

- A Python-based framework used to build interactive web applications and dashboards.
- **Implementation:** Used to create a real-time dashboard that displays live video feed, driver status (safe/drowsy), and alert notifications in a user-friendly interface.

Matplotlib

- A data visualization library used for plotting graphs and charts.
- **Implementation:** Used to visualize model performance metrics such as accuracy, loss graphs, and evaluation results.

HTML / CSS

- Markup and styling languages used for designing web interfaces.
- **Implementation:** Can be used to enhance the appearance of the dashboard and improve user interface design.

5.1.2 Backend Technologies

Python

- A high-level programming language widely used for machine learning, computer vision, and real-time applications.

- **Implementation:** Used as the core language for implementing the entire system, including video processing, model integration, and alert generation.

OpenCV

- An open-source computer vision library used for image and video processing.
- **Implementation:** Captures video from the camera, processes frames, detects faces, and performs preprocessing operations such as resizing and normalization.

TensorFlow

- Deep learning frameworks used to build and train neural network models.
- **Implementation:** Used to develop CNN models for eye state classification and yawning detection. These models help identify fatigue-related features in real time.

MediaPipe

- A framework for real-time face and landmark detection.
- **Implementation:** Detects facial landmarks such as eyes and mouth with high accuracy, which are used for feature extraction.

YOLO (You Only Look Once)

- A real-time object detection algorithm.
- **Implementation:** Used to detect mobile phone usage and identify driver distraction efficiently.

NumPy

- A library for numerical computations.
- **Implementation:** Used for handling arrays and performing mathematical operations on image data.

Pandas

- A data analysis library.
- **Implementation:** Used for handling and analyzing data during system evaluation and logging.

Visual Studio Code

- A lightweight and powerful code editor.
- **Implementation:** Used for writing, debugging, and managing project code with extensions for Python and machine learning.

Git & GitHub

- Version control system and repository platform.
- **Implementation:** Used to track code changes, manage versions, and collaborate efficiently.

Project Integration Example

1. **Input:** OpenCV captures live video from the camera.
2. **Processing:** MediaPipe and OpenCV detect face and extract features.
3. **Detection:** TensorFlow CNN models analyze eye state and yawning, while YOLO detects phone usage.
4. **Decision:** The system evaluates fatigue and distraction levels.
5. **Output:** Streamlit dashboard displays results and alerts the driver.

Operating System	Windows 10/11
Programming Language	Python
Code Editor	VS Code
Technologies	OpenCV, TensorFlow, MediaPipe, YOLO, Streamlit

Table 5.1 Software Specifications

5.2 Hardware Description

Processor

Intel Core i3 / i5 or higher

A multi-core processor is required to handle real-time video processing and deep learning computations. It supports:

- Continuous video frame processing
- Running AI models for detection
- Smooth execution of development tools

Higher processors (i5/i7) improve performance for real-time analysis.

RAM

Minimum 4GB (8GB recommended)

RAM is essential for handling:

- Real-time video processing
- Running machine learning models
- Executing development tools

8GB RAM is recommended for better performance and smooth multitasking.

Storage

At least 50GB free space

Storage is required for:

- Project files and datasets
- Model storage
- Temporary processing data

SSD storage improves speed and system performance.

Camera Module / Webcam

A camera is required to capture real-time video of the driver.

- Acts as the primary input device
- Should provide clear image quality for accurate detection

Graphics Card (Optional)

A GPU can accelerate deep learning model performance.

- Useful for training CNN models
- Improves real-time detection speed

Processor	Intel Core i3 / i5 or higher
Storage	At least 50GB free space
Camera	Webcam / Camera module
RAM	4GB (8GB recommended)
Graphics Card	Optional GPU
Internet connection	Stable connection (optional)

Table 5.2 Hardware Specification

5.3 Dataset Description

The performance of the proposed driver drowsiness detection system depends on the quality and diversity of the dataset used for training and testing. The dataset consists of images and video frames of drivers captured under different conditions, including variations in lighting, facial orientation, and driver behavior. The data is categorized into classes such as **eye open, eye closed, yawning, and non-yawning**, which are essential for identifying fatigue-related patterns.

The dataset includes publicly available image datasets along with manually collected samples to improve generalization. Each image is labeled based on the driver's condition, enabling supervised learning for accurate classification. The images are preprocessed by resizing, normalization, and format conversion to meet the input requirements of the deep learning models.

To enhance robustness, the dataset includes variations such as different head poses, facial expressions, and lighting conditions. Data augmentation techniques such as rotation, flipping, and brightness adjustment are applied to increase dataset diversity and reduce overfitting.

The dataset is divided into training and testing sets to evaluate model performance effectively. This structured dataset allows the system to learn meaningful features and ensures reliable detection of drowsiness in real-time driving conditions.

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 List of Modules

The implementation of the driver drowsiness detection system is carried out in a modular approach, where each module performs a specific task in the overall system. The system is divided into multiple modules starting from video acquisition to final testing and evaluation. Each module is designed to ensure efficient processing, accurate detection, and real-time performance.

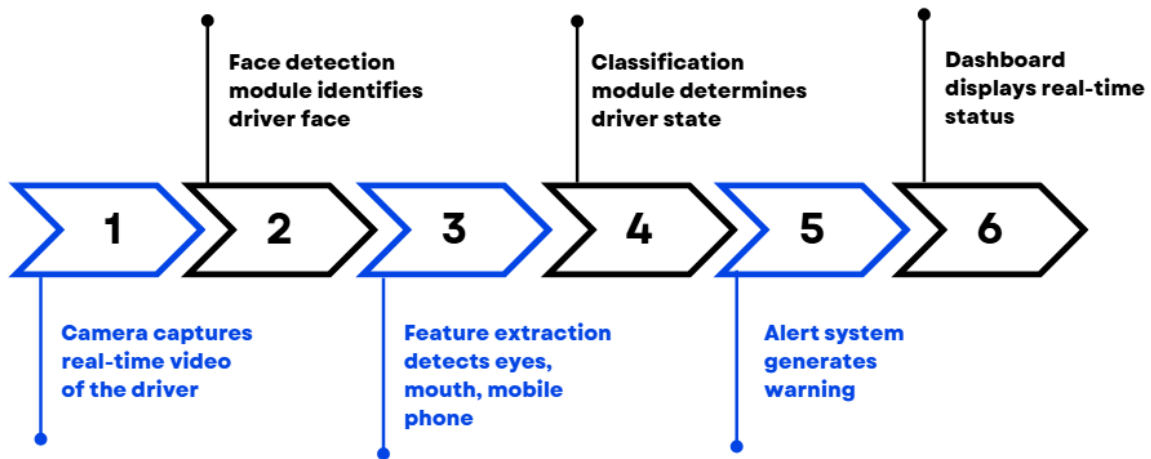


Fig 6.1 System Flow Diagram

6.1.1 Video Acquisition & Preprocessing

This module is responsible for capturing real-time video input from a webcam or in-vehicle camera. It is implemented using OpenCV, where the `cv2.VideoCapture()` function continuously reads frames in a loop. Each captured frame is resized to a fixed resolution to reduce computational load and improve processing speed. The color format is converted from BGR to RGB to match the input requirements of deep learning models.

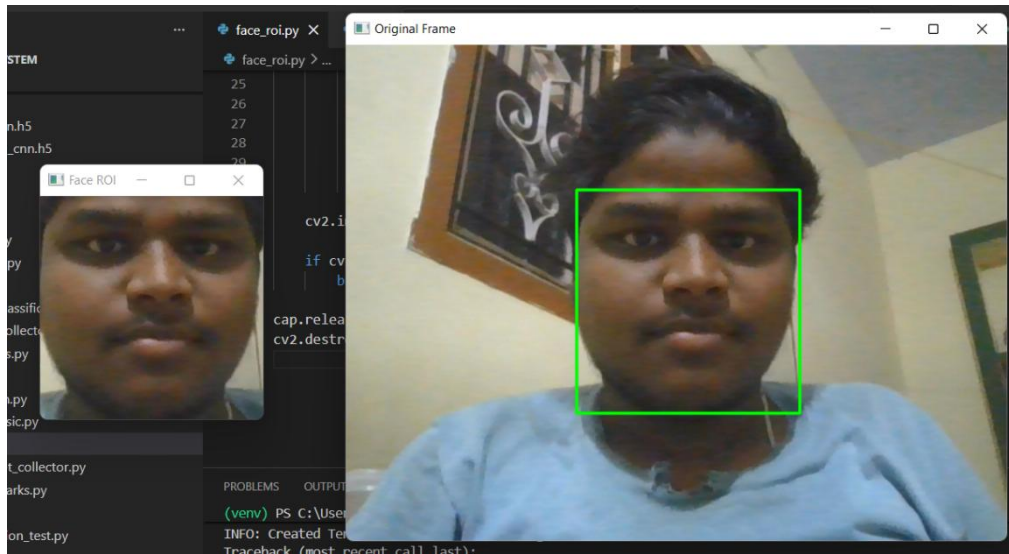


Fig 6.2 Video Acquisition & Preprocessing

Additional preprocessing steps such as normalization and frame rate control are applied to ensure smooth real-time performance. Error handling mechanisms are also included to manage camera failures or interruptions. This module ensures that clean and consistent frames are provided as input to subsequent modules, forming the foundation for accurate detection and analysis.

6.1.2 Face Detection & ROI Extraction

This module detects the driver's face in each frame and extracts the region of interest (ROI) for further processing. It is implemented using MediaPipe's face detection model, which efficiently identifies facial regions in real time. Once a face is detected, a bounding box is generated, and the corresponding region is cropped using OpenCV.

The system focuses only on the driver's face, ignoring background elements to reduce noise and computation. The extracted face is resized and normalized before passing it to the next stage. This module ensures that only relevant facial data is analyzed, improving accuracy and reducing unnecessary processing. It also handles slight variations in head movement and lighting conditions.

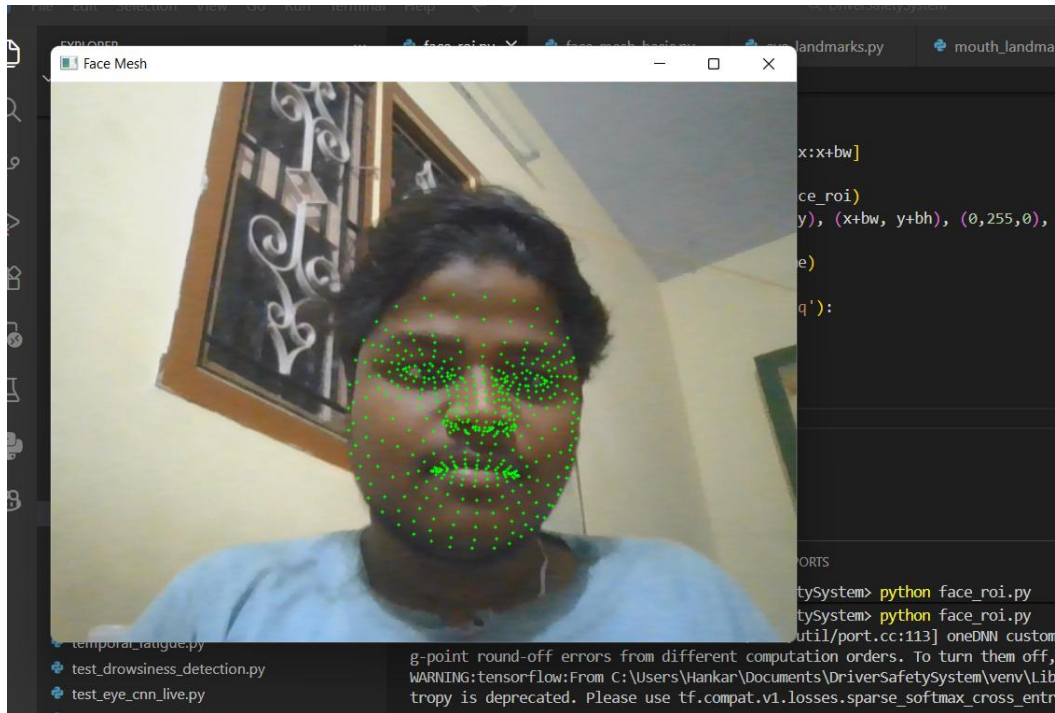


Fig 6.3 MediaPipe's Facial Landmarks

6.1.3 Eye & Mouth Region Detection

This module extracts specific facial features such as the eyes and mouth from the detected face. It uses MediaPipe's facial landmark detection to identify key points around these regions. Based on landmark coordinates, the eye and mouth areas are precisely cropped using OpenCV. The extracted regions are then resized and normalized to match the input size required by CNN models.

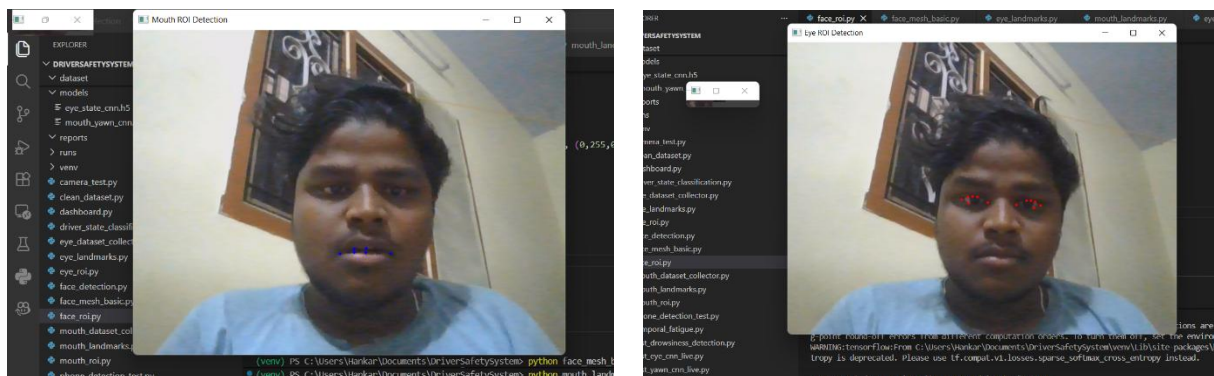


Fig 6.4 Eye and Mouth Region Extraction using MediaPipe

These regions serve as input for detecting eye closure and yawning. The module ensures accurate localization of features even under slight head movements. Proper extraction of these regions is critical, as it directly impacts the performance of the classification models used in later stages.

6.1.4 CNN-Based Eye State Classification

This module uses a Convolutional Neural Network (CNN) to classify the eye state as open or closed. The model is trained using labeled datasets containing eye images. During implementation, the extracted eye region is passed to the trained model, which predicts the eye state in real time. TensorFlow/Keras is used to build, train, and load the model.

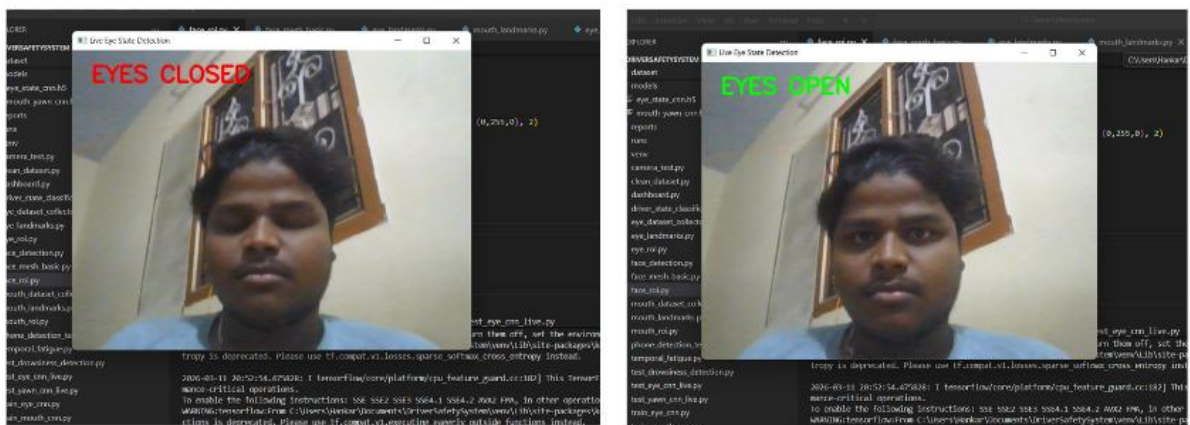


Fig 6.5 Eye State Classification

The system continuously monitors eye states across frames to detect prolonged closure. This module plays a key role in identifying drowsiness, as continuous eye closure is a strong indicator of fatigue. The use of CNN improves accuracy by automatically learning features from the data.

6.1.5 CNN-Based Yawning Detection

This module detects yawning behavior using a CNN model trained on mouth images. The extracted mouth region is passed to the trained model, which classifies it as yawning or non-yawning. TensorFlow/Keras is used for model

development and inference. The system tracks yawning occurrences over time to identify fatigue patterns.

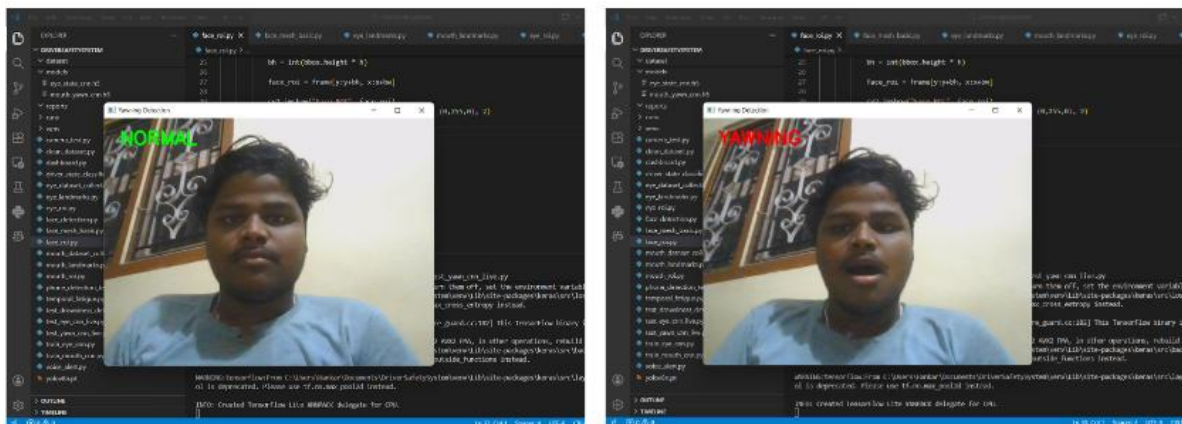


Fig 6.6 Mouth State Classification

Frequent yawning is considered a key indicator of drowsiness. This module enhances the reliability of the system by providing an additional fatigue indicator alongside eye detection. The use of deep learning ensures robust performance under different facial expressions and lighting conditions.

6.1.6 Temporal Fatigue Analysis

This module analyzes fatigue indicators over time rather than relying on single-frame predictions. It tracks eye closure duration and yawning frequency using logical conditions and time thresholds. For example, if the eyes remain closed for a specified duration, the system identifies it as drowsiness. Similarly, repeated yawning within a short period indicates fatigue.

This module is implemented using Python logic and counters that update continuously with each frame. Temporal analysis improves detection accuracy by reducing false positives and ensuring that transient events are not misclassified. It plays a crucial role in making the system more reliable and realistic.

6.1.7 Mobile Phone Distraction Detection

This module detects driver distraction caused by mobile phone usage. It is implemented using the YOLO (You Only Look Once) object detection model, which can identify objects in real time. The system processes each frame and checks for the presence of a mobile phone near the driver's face or hands. If a phone is detected, the system marks it as a distraction event.

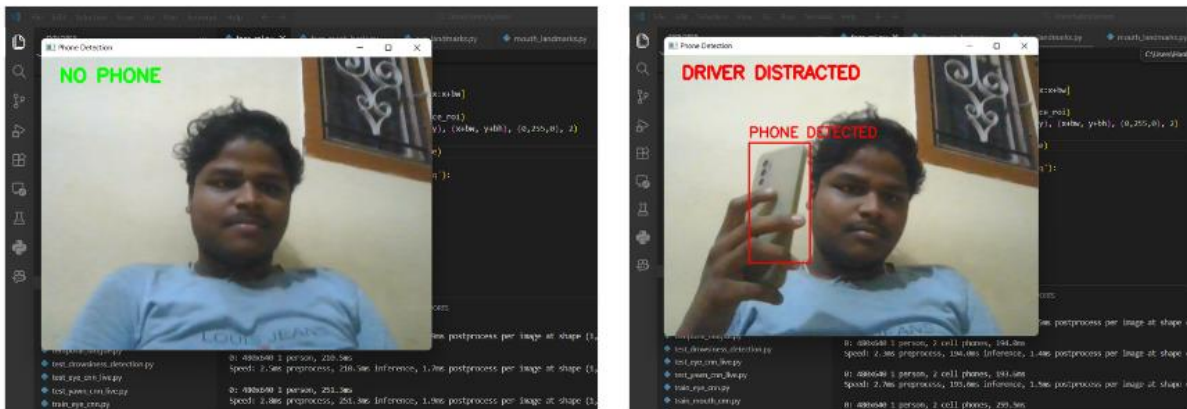


Fig 6.7 Mobile Phone Detection

YOLO provides fast and accurate detection, making it suitable for real-time applications. This module adds an important safety feature by monitoring driver attention, ensuring that the system addresses both fatigue and distraction.

6.1.8 Feature Fusion & Driver State Classification

This module combines the outputs from eye detection, yawning detection, and distraction detection to determine the driver's overall state. It uses rule-based logic or simple decision algorithms to classify the driver as safe, drowsy, or distracted.

For example, prolonged eye closure and frequent yawning may indicate drowsiness, while phone detection indicates distraction. The fusion of multiple indicators improves accuracy and reduces false alarms. This module acts as the

decision-making unit of the system, integrating information from different sources to provide a final classification.

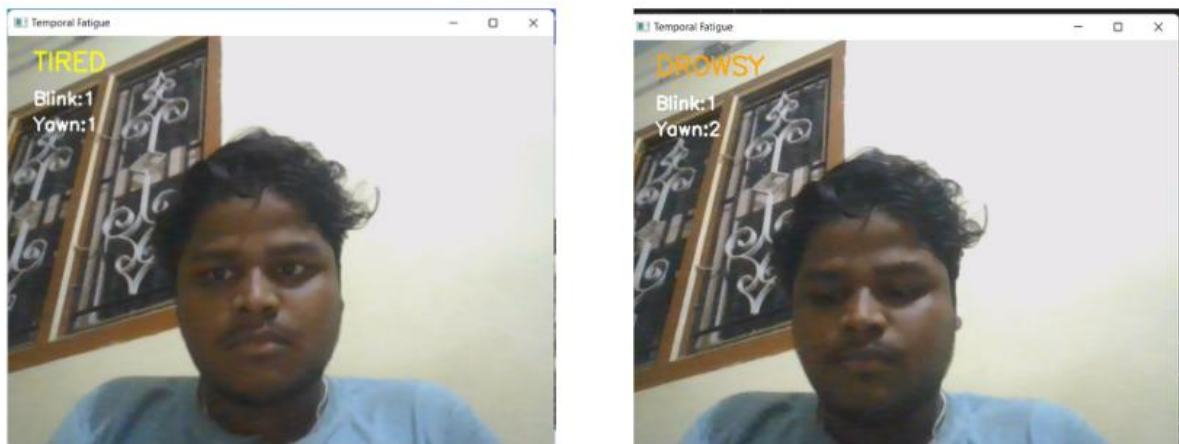


Fig 6.8 Driver State Classification

6.1.9 Alert System

This module generates alerts when unsafe driver behavior is detected. It is implemented using text-to-speech libraries such as `pyttsx3`, which convert warning messages into audio signals. When drowsiness or distraction is detected, the system triggers alerts like “You’re Tired, Take Rest”. The system uses thresholds to avoid excessive alerts and includes delays to prevent repetition.

This module ensures immediate feedback to the driver, helping prevent accidents. Optional hardware integration such as buzzers or vibration alerts can also be added for enhanced effectiveness.

6.1.10 Dashboard & Visualization

This module provides a user interface to display real-time monitoring results. It is implemented using `Streamlit`, which allows the creation of interactive dashboards using Python. The dashboard shows live video, driver state, and alert messages. It updates continuously to reflect real-time changes.

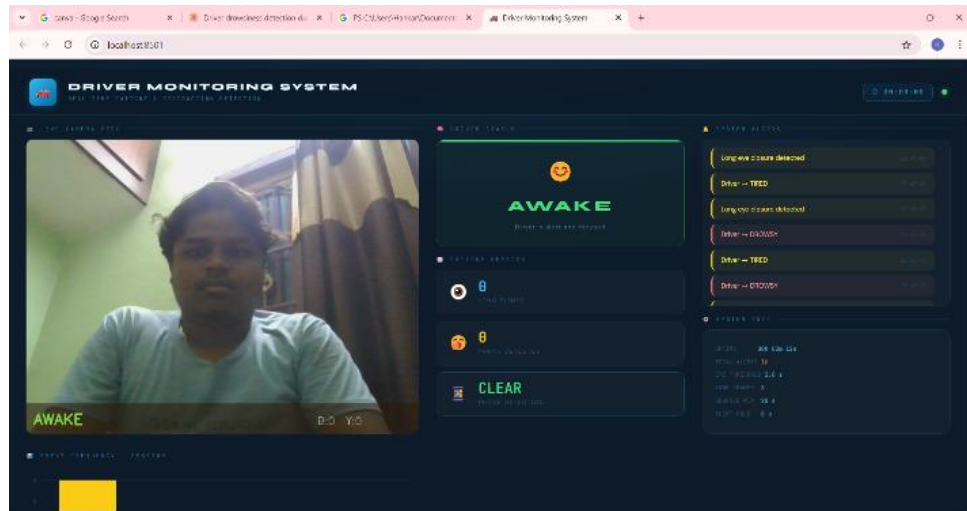


Fig 6.9 Dashboard

Additional features such as logs and performance metrics can also be displayed. This module improves usability by providing clear and visual feedback, making the system easier to monitor and understand.

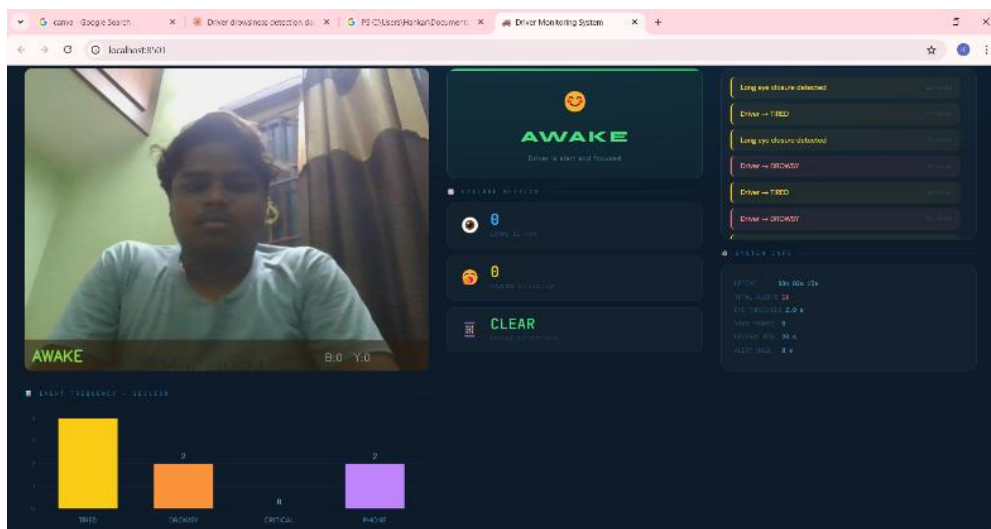


Fig 6.10 Dashboard

6.1.11 System Integration & Optimization

This module integrates all individual modules into a single working pipeline. It ensures smooth data flow between modules and optimizes performance for real-time execution. Techniques such as reducing processing delay, optimizing model inference time, and minimizing redundant computations

are applied. The system is tested for stability and efficiency under continuous operation. This module ensures that the entire system works seamlessly as a unified application.

6.1.12 Testing, Evaluation & Documentation

This module involves testing the system under different conditions to evaluate its performance. Various scenarios such as normal driving, drowsiness, and distraction are tested. Metrics like accuracy, response time, and robustness are analyzed.

The system is validated using both real-time input and datasets. Documentation is prepared, including reports, results, and system descriptions. This module ensures that the system meets performance requirements and is ready for deployment

CHAPTER 7

TESTING

7.1 Testing

The testing phase of the driver drowsiness detection system is carried out to evaluate the performance, accuracy, and reliability of the developed model under real-time conditions. The trained CNN models for eye state and yawning detection are tested using both recorded datasets and live video input to ensure consistent behavior. Various test cases are designed to simulate real driving scenarios such as normal driving, eye closure, yawning, and head movement. The system is also tested under different lighting conditions and facial orientations to verify robustness. Each functionality, including video capture, face detection, feature extraction, classification, and alert generation, is validated individually and as part of the integrated system. The expected outcomes are compared with actual outputs to identify errors or mismatches. Performance metrics such as detection accuracy and response time are observed to ensure real-time operation. The testing results confirm that the system effectively detects drowsiness and provides timely alerts, demonstrating its suitability for practical deployment.

7.1.1 Test Cases

S.No	Functionality	Expected Outcome	Actual Outcome	Result
1	Video Capture	Camera captures live video continuously	Video captured without interruption	Pass
2	Face Detection	System detects driver face correctly	Face detected accurately in most frames	Pass

3	Eye Detection	Eye region extracted properly	Eye region detected correctly	Pass
4	Eye State Classification	Correctly classify open/closed eyes	Accurate classification observed	Pass
5	Yawning Detection	Detect yawning condition	Yawning detected correctly	Pass
6	Drowsiness Detection	Identify drowsy state based on thresholds	Drowsiness detected with good accuracy	Pass
7	Alert System	Generate alert when drowsy	Audio alert triggered correctly	Pass
8	Real-time Performance	System runs without delay	Smooth real-time execution observed	Pass
9	Low Light Condition	System performs under low lighting	Slight accuracy drop but functional	Pass
10	Mobile Phone Detection	Detects the usage of mobile phone	Mobile phone is detected	Pass

Table 7.1 Test Cases

CHAPTER 8

RESULTS AND PERFORMANCE COMPARISON

8.1 Detection Efficiency

The proposed driver drowsiness detection system demonstrates high detection efficiency by accurately identifying fatigue-related behaviors such as eye closure and yawning. The CNN-based models used for eye state and yawning detection achieve very high accuracy, ensuring reliable performance in real-time conditions. The eye detection model achieved an accuracy of 99.60%, while the yawning detection model achieved 99.33%, indicating strong classification capability.

The confusion matrices show minimal misclassification, confirming that the models are highly precise in distinguishing between normal and drowsy states. The system effectively processes continuous video frames and maintains consistency in detection across different conditions. By combining multiple indicators and temporal analysis, the system improves overall detection reliability and reduces false alarms, making it suitable for real-world deployment.

8.2 Scalability Evaluation

The system is designed to be scalable and adaptable for various real-world applications. It can operate efficiently on standard computing devices such as laptops without requiring high-end hardware. The use of optimized libraries like OpenCV and MediaPipe ensures fast processing and low computational overhead. The modular design allows easy integration of additional features such as head pose detection or cloud-based monitoring.

The system can also be extended to support multiple cameras or deployed in different vehicle environments. Since the models are lightweight and optimized, the system maintains real-time performance even with continuous

input streams. This scalability makes the system suitable for both individual use and large-scale deployment in intelligent transportation systems.

8.3 Performance Metrics

The performance of the system is evaluated using standard metrics such as accuracy, precision, recall, and F1-score. The eye state detection model achieved an accuracy of **99.60%**, while the yawning detection model achieved **99.33%**, indicating excellent classification performance. Precision and recall values are close to 1.0, showing that the models effectively minimize both false positives and false negatives.

The confusion matrices further validate the reliability of the models, with very few misclassifications. The high F1-score confirms a balanced performance between precision and recall. These results demonstrate that the system is highly accurate and capable of performing real-time detection with minimal errors.

Metric	Value	Notes
Eye Detection Accuracy	99.60%	High accuracy with minimal false classification
Yawning Detection Accuracy	99.33%	Reliable detection of yawning behavior
Precision	~0.99	Very low false positives
Recall	~0.99	High detection rate of true positives
F1-Score	~0.99	Balanced model performance
Response Time	Real-time	Processes frames without delay

Table 8.1 Performance Metrics

8.4 Comparison with Existing System

The proposed system is compared with traditional driver monitoring approaches, including driver performance-based, physiological signal-based, and vision-based systems. Driver performance-based systems rely on vehicle behavior and fail to detect early-stage fatigue. Physiological systems provide accurate results but require intrusive sensors. Traditional vision-based systems use handcrafted features and are sensitive to environmental conditions. In contrast, the proposed system uses deep learning and multiple indicators, providing improved accuracy, real-time performance, and non-intrusive monitoring.

Feature	Driver Performance-Based System	Physiological System	Vision-Based System	Proposed System
Detection Method	Vehicle behavior	Biological signals	Image processing	Deep learning (CNN)
Intrusiveness	Non-intrusive	Intrusive	Non-intrusive	Non-intrusive
Accuracy	Moderate	High	Moderate	Very High
Real-time Detection	Limited	Limited	Moderate	High
Early Detection	Poor	Good	Moderate	Excellent

Cost	Medium	High	Low	Low
Robustness	Low	High	Moderate	High

Table 8.2 Comparison with Existing System

8.5 Operational Efficiency Gains

The proposed system offers significant improvements in operational efficiency compared to traditional approaches. By using real-time video processing and deep learning models, the system provides fast and accurate detection without requiring additional hardware such as sensors. The non-intrusive design ensures ease of use and better user acceptance. The integration of multiple detection features improves reliability and reduces false alarms.

Additionally, the system operates efficiently with minimal computational resources, making it suitable for deployment on standard devices. Real-time alert generation ensures immediate driver response, which helps in preventing accidents. Overall, the system enhances safety, reduces complexity, and provides a cost-effective solution for driver monitoring.

CHAPTER 9

CONCLUSION AND FUTURE WORK

9.1 Conclusion

The driver drowsiness detection system developed in this project provides an effective approach for monitoring driver alertness and identifying signs of fatigue and distraction in real time. By using computer vision and deep learning techniques, the system analyzes facial features such as eye closure, yawning, and mobile phone usage to determine the driver's condition. The ability to detect early symptoms of drowsiness and generate timely alerts helps in preventing unsafe driving situations and reduces the risk of accidents.

The proposed system is non-intrusive, cost-effective, and suitable for real-world implementation as it does not require wearable sensors or complex hardware. The use of multiple fatigue indicators improves detection accuracy and reliability compared to traditional methods. Overall, this project demonstrates that intelligent driver monitoring systems can play an important role in enhancing road safety and supporting safer driving practices.

9.2 Future Work

The proposed driver drowsiness detection system can be further enhanced by integrating advanced technologies to improve accuracy, adaptability, and real-world usability. While the current system effectively identifies fatigue and distraction using visual features, future improvements can focus on handling complex driving conditions and enabling smarter decision-making. One key enhancement is the integration with vehicle control systems, allowing automatic safety actions such as speed adjustment or braking during critical situations. Predictive fatigue analysis can also be introduced to detect early signs of tiredness by analyzing long-term behavioral patterns, enabling preventive alerts. Additionally, driver personalization can help the system adapt to individual behavior, reducing false alarms and improving reliability. Expanding distraction detection to include gaze direction, head posture, and hand movements can provide a more comprehensive understanding of driver attention. These advancements will make the system more intelligent, efficient, and suitable for practical deployment.

APPENDIX - I

SOURCE CODE

train_eye_cnn.py

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
IMG_SIZE = 64
BATCH_SIZE = 32
EPOCHS = 15
# Data generator
datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)
train_data = datagen.flow_from_directory(
    "dataset/eye",
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="binary",
    subset="training"
)
val_data = datagen.flow_from_directory(
    "dataset/eye",
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="binary",
    subset="validation"
)
# CNN Model
model = Sequential([
    Conv2D(32, (3,3), activation="relu", input_shape=(64,64,3)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation="relu"),
    MaxPooling2D(2,2),
    Conv2D(128, (3,3), activation="relu"),
    MaxPooling2D(2,2),
    Flatten(),
```

```

        Dense(128, activation="relu"),
        Dropout(0.5),
        Dense(1, activation="sigmoid")
    ])
    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    model.summary()
    # Train
    model.fit(
        train_data,
        validation_data=val_data,
        epochs=EPOCHS
    )
    # Save model
    model.save("models/eye_state_cnn.h5")
    print("Eye state CNN model saved successfully")

```

train_yawn_cnn.py

```

import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
Dropout
import os
IMG_SIZE = 64
BATCH = 32
EPOCHS = 15
train_dir = "dataset/mouth"
datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)
train_data = datagen.flow_from_directory(
    train_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH,
    class_mode='binary',
    subset='training'
)

```

```

val_data = datagen.flow_from_directory(
    train_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH,
    class_mode='binary',
    subset='validation'
)
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(IMG_SIZE, IMG_SIZE,
3)),
    MaxPooling2D(),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(),
    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
model.summary()
model.fit(train_data, validation_data=val_data, epochs=EPOCHS)
os.makedirs("models", exist_ok=True)
model.save("models/mouth_yawn_cnn.h5")
print("Mouth yawning CNN model saved!")

```

test_eye_cnn_live.py

```

import cv2
import mediapipe as mp
import numpy as np
import tensorflow as tf
# Load trained CNN model
model = tf.keras.models.load_model("models/eye_state_cnn.h5")
# Eye landmark indices
LEFT_EYE = [33, 133, 160, 159, 158, 144]
RIGHT_EYE = [362, 263, 387, 386, 385, 373]

```

```

padding = 16
IMG_SIZE = 64
mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(refine_landmarks=True,
max_num_faces=1)
cap = cv2.VideoCapture(0)
def predict_eye_state(eye_img):
    eye_img = cv2.resize(eye_img, (IMG_SIZE, IMG_SIZE))
    eye_img = eye_img / 255.0
    eye_img = np.expand_dims(eye_img, axis=0)
    pred = model.predict(eye_img, verbose=0)[0][0]
    return "OPEN" if pred > 0.5 else "CLOSED"
while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.flip(frame, 1)
    h, w, _ = frame.shape
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = face_mesh.process(rgb)
    state_text = "Detecting..."
    color = (255, 255, 0)
    if results.multi_face_landmarks:
        for face_landmarks in results.multi_face_landmarks:
            # LEFT EYE
            lx = [int(face_landmarks.landmark[i].x * w) for i in LEFT_EYE]
            ly = [int(face_landmarks.landmark[i].y * h) for i in LEFT_EYE]
            lx1 = max(min(lx) - padding, 0)
            lx2 = min(max(lx) + padding, w)
            ly1 = max(min(ly) - padding, 0)
            ly2 = min(max(ly) + padding, h)
            left_eye = frame[ly1:ly2, lx1:lx2]
            # RIGHT EYE
            rx = [int(face_landmarks.landmark[i].x * w) for i in RIGHT_EYE]
            ry = [int(face_landmarks.landmark[i].y * h) for i in RIGHT_EYE]
            rx1 = max(min(rx) - padding, 0)
            rx2 = min(max(rx) + padding, w)
            ry1 = max(min(ry) - padding, 0)
            ry2 = min(max(ry) + padding, h)
            right_eye = frame[ry1:ry2, rx1:rx2]
            if left_eye.size != 0 and right_eye.size != 0:
                left_state = predict_eye_state(left_eye)

```

```

        right_state = predict_eye_state(right_eye)
        if left_state == "CLOSED" and right_state == "CLOSED":
            state_text = "EYES CLOSED"
            color = (0, 0, 255)
        else:
            state_text = "EYES OPEN"
            color = (0, 255, 0)
    cv2.putText(frame, state_text, (30, 50),
                cv2.FONT_HERSHEY_SIMPLEX, 1.2, color, 3)
    cv2.imshow("Live Eye State Detection", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()

```

test_yawn_cnn_live.py

```

import cv2
import mediapipe as mp
import numpy as np
import tensorflow as tf
model = tf.keras.models.load_model("models/mouth_yawn_cnn.h5")
MOUTH = [61, 291, 0, 17, 78, 308]
IMG_SIZE = 64
padding = 20
mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(refine_landmarks=True,
max_num_faces=1)
cap = cv2.VideoCapture(0)
def predict_mouth(img):
    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
    img = img / 255.0
    img = np.expand_dims(img, axis=0)
    pred = model.predict(img, verbose=0)[0][0]
    return "YAWNING" if pred > 0.5 else "NORMAL"
while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.flip(frame, 1)
    h, w, _ = frame.shape
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = face_mesh.process(rgb)

```

```

status = "Detecting..."
if results.multi_face_landmarks:
    for face_landmarks in results.multi_face_landmarks:
        mx = [int(face_landmarks.landmark[i].x * w) for i in MOUTH]
        my = [int(face_landmarks.landmark[i].y * h) for i in MOUTH]
        x1 = max(min(mx) - padding, 0)
        x2 = min(max(mx) + padding, w)
        y1 = max(min(my) - padding, 0)
        y2 = min(max(my) + padding, h)
        mouth = frame[y1:y2, x1:x2]
        if mouth.size != 0:
            status = predict_mouth(mouth)
color = (0,0,255) if status=="YAWNING" else (0,255,0)
cv2.putText(frame, status, (30,50),
            cv2.FONT_HERSHEY_SIMPLEX, 1.2, color, 3)
cv2.imshow("Yawning Detection", frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break
cap.release()
cv2.destroyAllWindows()

```

phone_detection.py

```

import cv2
from ultralytics import YOLO
# Load YOLO model
model = YOLO("yolov8n.pt")
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    results = model(frame)
    phone_detected = False
    for r in results:
        boxes = r.boxes
        for box in boxes:
            cls = int(box.cls[0])
            label = model.names[cls]
            if label == "cell phone":
                phone_detected = True
                x1,y1,x2,y2 = map(int, box.xyxy[0])
                cv2.rectangle(frame,(x1,y1),(x2,y2),(0,0,255),2)

```



```

        cv2.putText(frame,"PHONE DETECTED",(x1,y1-10),
                    cv2.FONT_HERSHEY_SIMPLEX,0.8,(0,0,255),2)
    if phone_detected:
        cv2.putText(frame,"DRIVER DISTRACTED",(30,40),
                    cv2.FONT_HERSHEY_SIMPLEX,1,(0,0,255),3)
    else:
        cv2.putText(frame,"NO PHONE",(30,40),
                    cv2.FONT_HERSHEY_SIMPLEX,1,(0,255,0),3)
    cv2.imshow("Phone Detection",frame)
    if cv2.waitKey(1)==ord("q"):
        break
cap.release()
cv2.destroyAllWindows()

```

voice_alert.py

```

import pyttsx3
import time
import threading
last_spoken_time = 0
COOLDOWN = 6 # seconds between alerts
def speak_async(text):
    def run():
        engine = pyttsx3.init()
        engine.setProperty("rate",160)
        engine.say(text)
        engine.runAndWait()
        engine.stop()
    t = threading.Thread(target=run)
    t.daemon = True
    t.start()
def driver_alert(state, phone_detected):
    global last_spoken_time
    now = time.time()
    # prevent speaking too frequently
    if now - last_spoken_time < COOLDOWN:
        return
    if phone_detected:
        speak_async("Don't use mobile phone while driving")
        last_spoken_time = now
        return
    if state == "TIRED":
        speak_async("You are tired. Please take rest")

```

```

        last_spoken_time = now
    elif state == "DROWSY":
        speak_async("You are very tired. Take a break")
        last_spoken_time = now
    elif state == "CRITICAL":
        speak_async("Warning. You are extremely tired. Stop the vehicle immediately")
        last_spoken_time = now

```

driver_state_classification.py

```

import cv2
import mediapipe as mp
import numpy as np
import tensorflow as tf
import time
from ultralytics import YOLO
from voice_alert import driver_alert
# ===== LOAD MODELS =====
eye_model = tf.keras.models.load_model("models/eye_state_cnn.h5")
mouth_model = tf.keras.models.load_model("models/mouth_yawn_cnn.h5")
yolo_model = YOLO("yolov8n.pt")
LEFT_EYE = [33,133,160,159,158,144]
MOUTH = [61,291,0,17,78,308]
IMG = 64
padding = 16
EYE_TIME_THRESHOLD = 2.0
MOUTH_FRAMES = 8
OBSERVE_TIME = 20
ALERT_HOLD = 8
eye_close_start = None
eye_event_triggered = False
long_blinks = 0
yawn_count = 0
window_start = None
alert_start = None
state = "AWAKE"
mouth_open_frames = 0
mp_face_mesh = mp.solutions.face_mesh
face_mesh =
mp_face_mesh.FaceMesh(refine_landmarks=True,max_num_faces=1)
cap = cv2.VideoCapture(0)
def preprocess(img):

```

```

img=cv2.resize(img,(IMG,IMG))
img=img/255.0
return np.expand_dims(img,0)
while True:
    ret,frame=cap.read()
    if not ret:
        break
    frame=cv2.flip(frame,1)
    h,w,_=frame.shape
    rgb=cv2.cvtColor(frame,cv2.COLOR_BGR2RGB)
    results=face_mesh.process(rgb)
    eye_closed=False
    yawning=False
    now=time.time()
    # ===== MODULE 6 (UNCHANGED) =====
    if results.multi_face_landmarks:
        lm=results.multi_face_landmarks[0]
        ex=[int(lm.landmark[i].x*w) for i in LEFT_EYE]
        ey=[int(lm.landmark[i].y*h) for i in LEFT_EYE]
        eye=frame[min(ey)-padding:max(ey)+padding, min(ex)-padding:max(ex)+padding]
        mx=[int(lm.landmark[i].x*w) for i in MOUTH]
        my=[int(lm.landmark[i].y*h) for i in MOUTH]
        mouth=frame[min(my)-padding:max(my)+padding, min(mx)-padding:max(mx)+padding]
        if eye.size:
            p=eye_model.predict(preprocess(eye),verbose=0)[0][0]
            eye_closed = p<0.5
        if mouth.size:
            m=mouth_model.predict(preprocess(mouth),verbose=0)[0][0]
            yawning = m<0.5
    # ----- EYE TEMPORAL -----
    if eye_closed:
        if eye_close_start is None:
            eye_close_start = now
        if now-eye_close_start>=EYE_TIME_THRESHOLD and not eye_event_triggered:
            long_blinks+=1
            eye_event_triggered=True
            print("Long eye closure")
    else:
        eye_close_start=None
        eye_event_triggered=False

```

```

# ----- YAWN -----
if yawning:
    mouth_open_frames+=1
else:
    if mouth_open_frames>=MOUTH_FRAMES:
        yawn_count+=1
        print("Yawn")
        mouth_open_frames=0
# ----- START WINDOW -----
if window_start is None and (long_blinks>0 or yawn_count>0):
    window_start=now
# ----- DECISION -----
if window_start:
    if now-window_start>OBSERVE_TIME:
        long_blinks=0
        yawn_count=0
        window_start=None
        state="AWAKE"
    if long_blinks>=1 or yawn_count>=2:
        state="TIRED"
        alert_start=now
    if long_blinks>=1 and yawn_count>=2:
        state="DROWSY"
        alert_start=now
    if long_blinks>=3:
        state="CRITICAL"
        alert_start=now
# ----- ALERT HOLD -----
if alert_start and now-alert_start>ALERT_HOLD:
    alert_start=None
    window_start=None
    long_blinks=0
    yawn_count=0
    state="AWAKE"
# ===== MODULE 7 (PHONE DETECTION) =====
phone_detected = False
results_yolo = yolo_model(frame)
for r in results_yolo:
    for box in r.bboxes:
        cls = int(box.cls[0])
        label = yolo_model.names[cls]
        if label == "cell phone":
            phone_detected = True

```

```

        x1,y1,x2,y2 = map(int,box.xyxy[0])
        cv2.rectangle(frame,(x1,y1),(x2,y2),(0,0,255),2)
        cv2.putText(frame,"PHONE",(x1,y1-10),
                    cv2.FONT_HERSHEY_SIMPLEX,0.8,(0,0,255),2)
# ===== DISPLAY =====
color=(0,255,0)
if state=="TIRED": color=(0,255,255)
if state=="DROWSY": color=(0,165,255)
if state=="CRITICAL": color=(0,0,255)
cv2.putText(frame,state,(30,40),
            cv2.FONT_HERSHEY_SIMPLEX,1,color,2)
cv2.putText(frame,f"Blink: {long_blinks}",(30,80),0,0.7,(255,255,255),2)
cv2.putText(frame,f"Yawn: {yawn_count}",(30,110),0,0.7,(255,255,255),2)
# ----- MOBILE WARNING -----
if phone_detected:
    cv2.putText(frame,"DON'T USE MOBILE PHONE",(30,150),
                cv2.FONT_HERSHEY_SIMPLEX,1,(0,0,255),3)
driver_alert(state, phone_detected)
cv2.imshow("Driver Monitoring System",frame)
if cv2.waitKey(1)==ord("q"):
    break
cap.release()
cv2.destroyAllWindows()

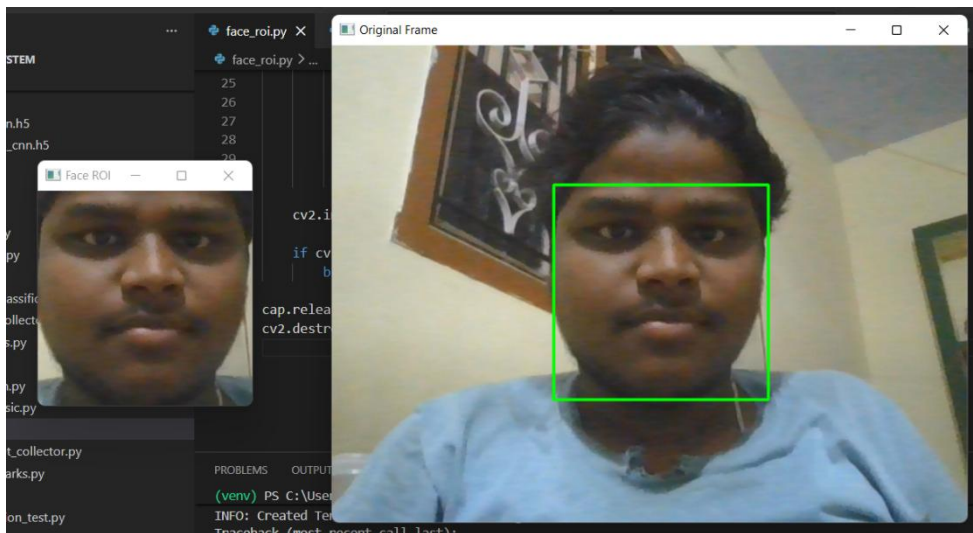
```

APPENDIX - II

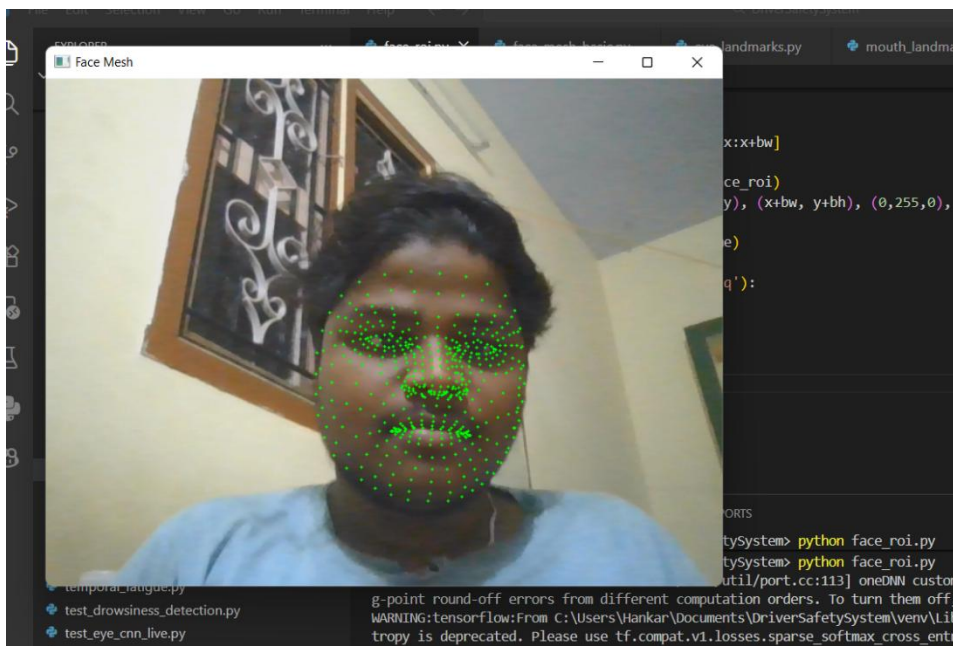
SCREENSHOTS

1. Face Detection Output

This screenshot shows the detection of the driver's face within the video frame. A bounding box is drawn around the detected face, indicating that the system has successfully identified the region of interest. This step ensures that only the facial region is processed for further analysis.



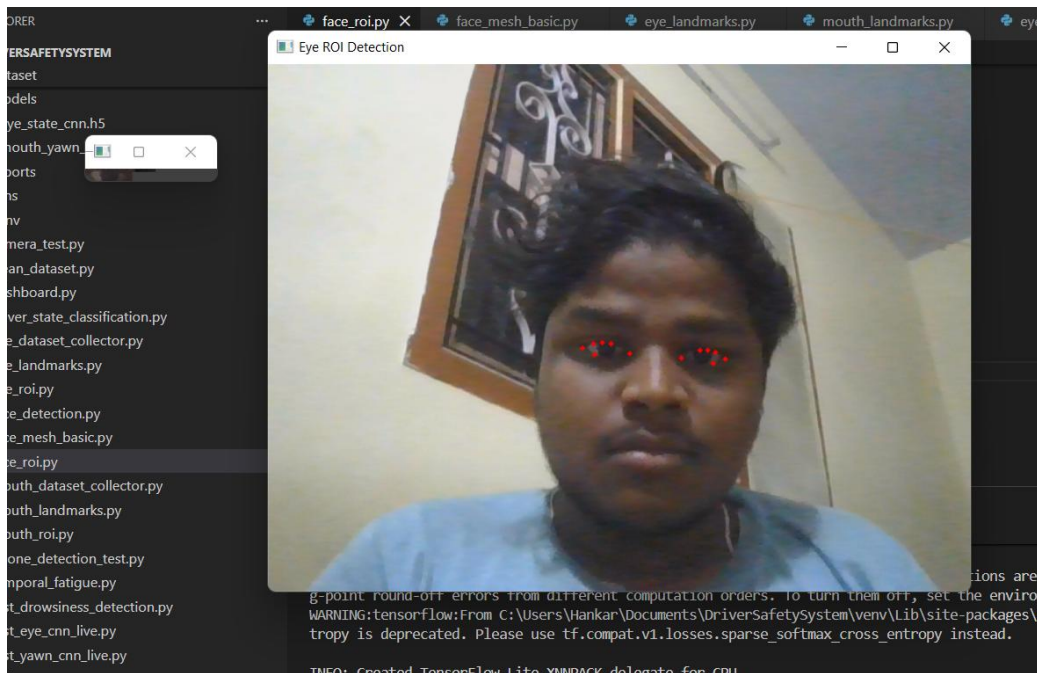
2. Eye Region Extraction



This screenshot shows the cropped eye region obtained from the detected face. The system isolates the eye area to analyze eye state (open or closed). Proper extraction of this region is important for accurate classification by the deep learning model.

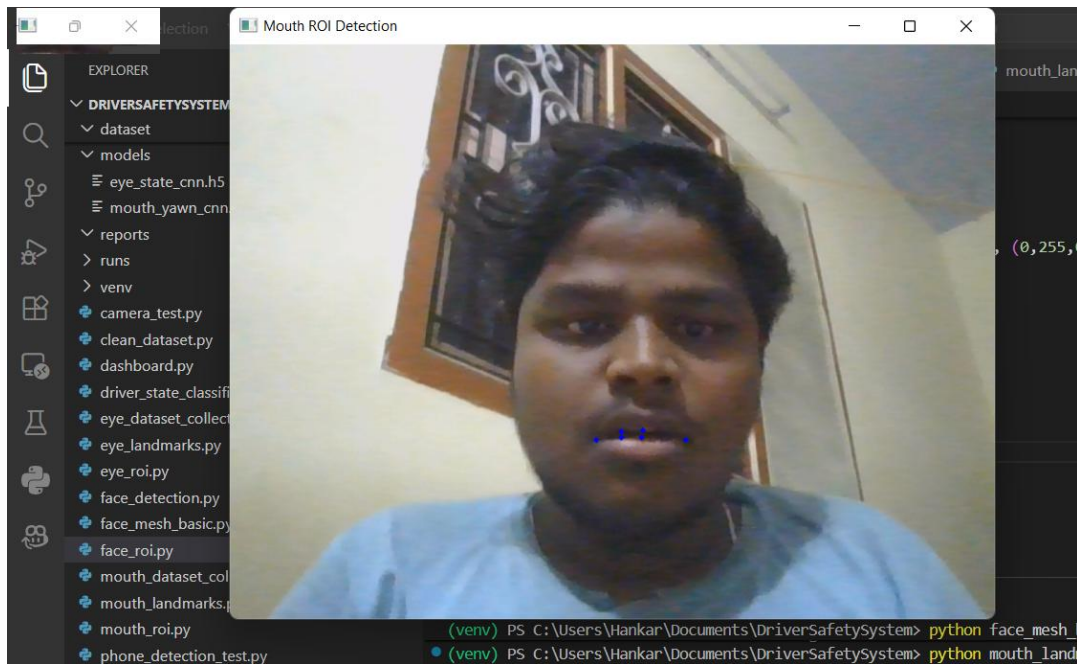
3. Eye Region Extraction

This screenshot shows the cropped eye region obtained from the detected face. The system isolates the eye area to analyze eye state (open or closed). Proper extraction of this region is important for accurate classification by the deep learning model.



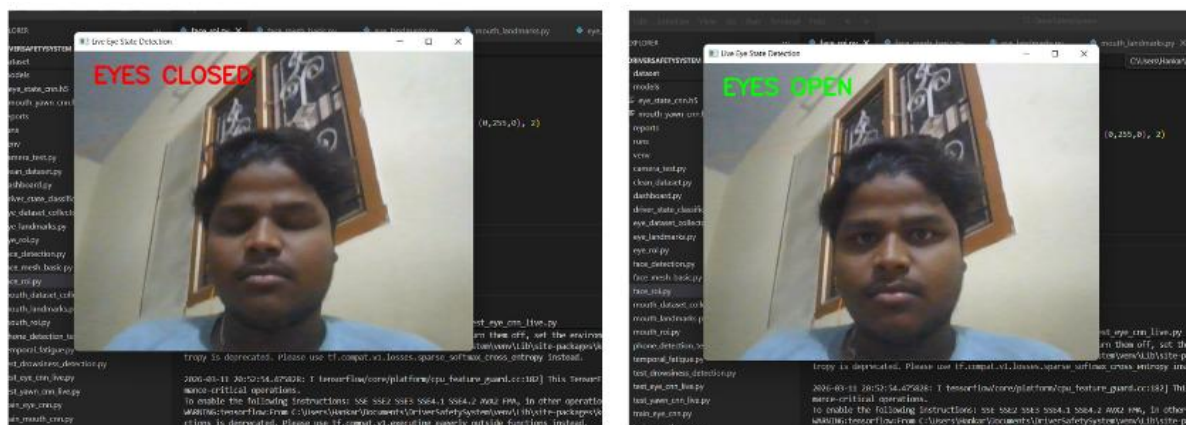
4. Mouth Region Extraction

This screenshot displays the extracted mouth region used for yawning detection. The system identifies and isolates the mouth area to determine whether the driver is yawning. This helps in improving fatigue detection accuracy.



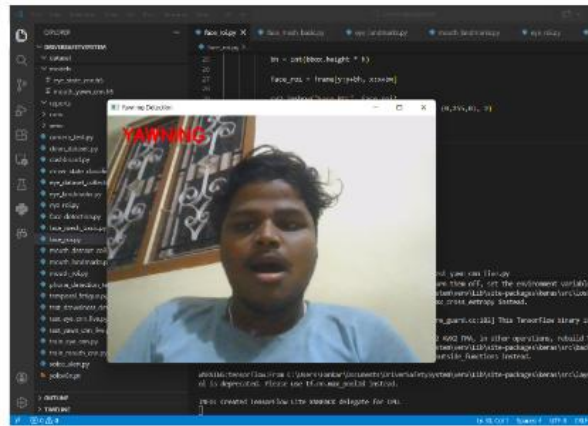
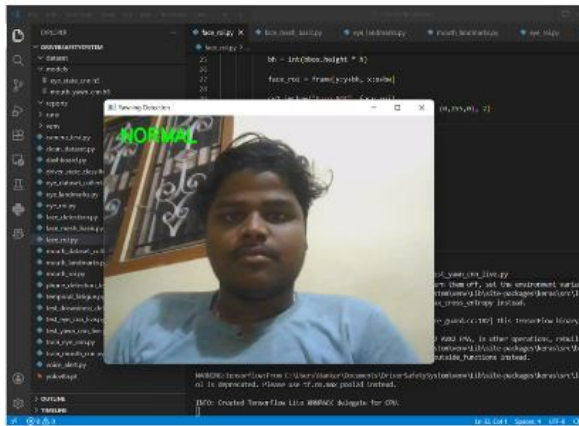
5. Eye State Detection Output

This screenshot shows the classification result of the eye detection model. The system identifies whether the driver's eyes are open or closed and displays the result on the screen. Accurate prediction confirms the effectiveness of the trained CNN model.



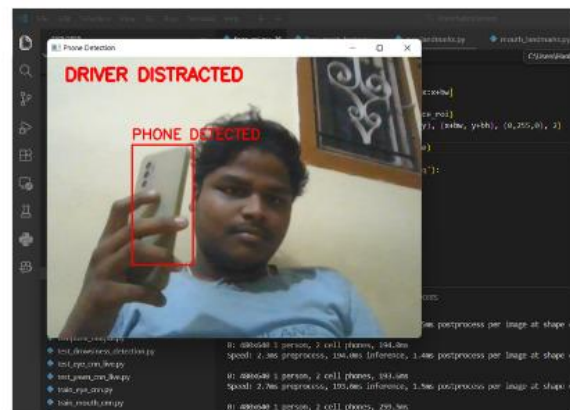
6. Yawning Detection Output

This screenshot presents the output of the yawning detection model. The system detects whether the driver is yawning and displays the corresponding result. This feature helps in identifying signs of fatigue.



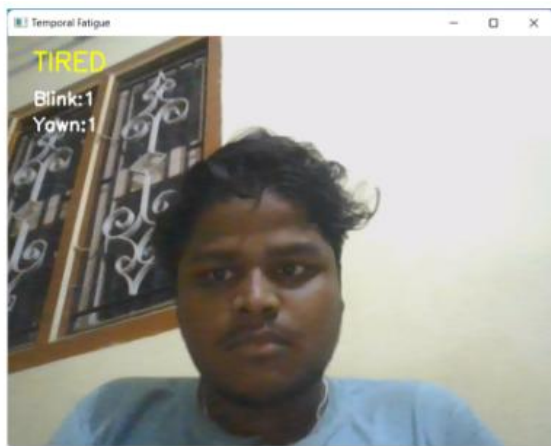
7. Mobile Phone Detection

This screenshot presents the output of the mobile phone detection model. The system detects whether the driver is using mobile phone and displays the corresponding result. This feature helps in identifying signs of distraction.



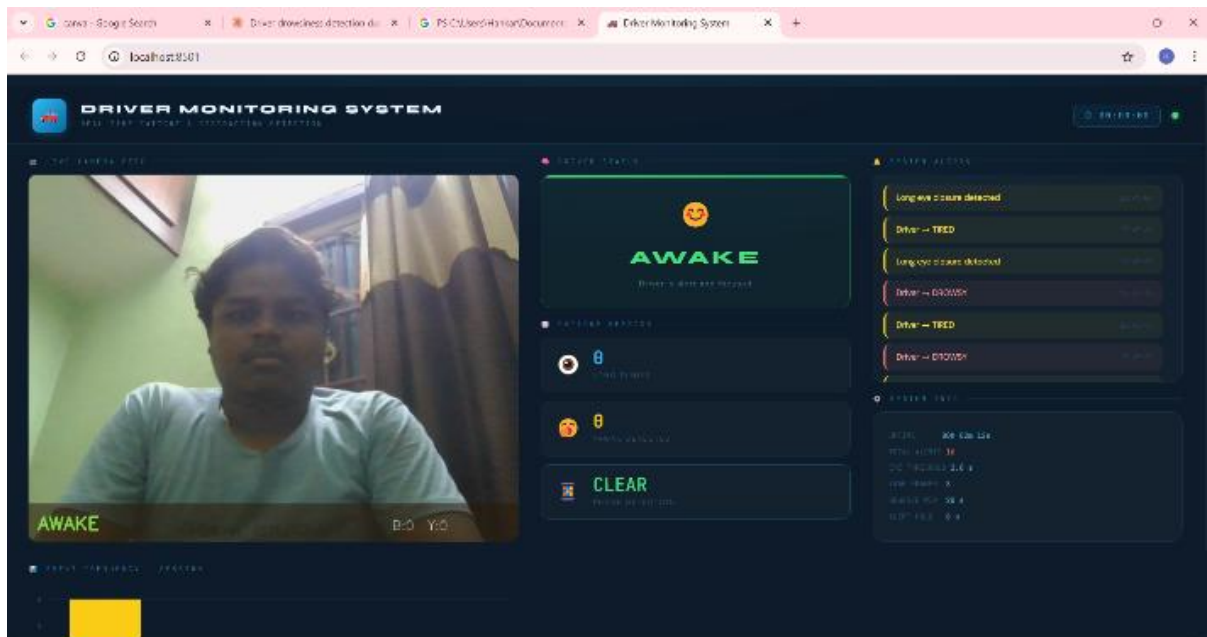
8. Drowsiness Detection Output

This screenshot shows the final driver state classification based on combined indicators. The system evaluates eye closure and yawning patterns to determine whether the driver is in a drowsy state. The result is displayed clearly for user understanding.



9. Dashboard Visualization

This screenshot shows the dashboard interface displaying real-time driver status. It provides a clear view of the system output, including driver state and alerts. The dashboard enhances usability and monitoring.



10. System Performance Output

This screenshot presents the model performance results such as accuracy or confusion matrix. It demonstrates the effectiveness of the trained models in detecting drowsiness with high accuracy.

```

===== Eye State Detection Model =====
WARNING:tensorflow:From C:\Users\Hankar\Documents\DriverSafetySystem\venv\Lib\site-packages\keras\src\utils\tf_utils.py:110: tf.nn.conv2d is deprecated. Please use tf.compat.v1.nn.conv2d instead.

48/48 [=====] - 26s 407ms/step - loss: 0.0170 - accuracy: 0.9961
Eye Model Accuracy: 99.6073305606842 %
48/48 [=====] - 3s 56ms/step

Confusion Matrix (Eye Model)
[[746   0]
 [   6 776]]

Classification Report (Eye Model)

```

	precision	recall	f1-score	support
0	0.99	1.00	1.00	746
1	1.00	0.99	1.00	782
accuracy			1.00	1528
macro avg	1.00	1.00	1.00	1528
weighted avg	1.00	1.00	1.00	1528

```

===== Yawning Detection Model =====
19/19 [=====] - 8s 413ms/step - loss: 0.0098 - accuracy: 0.9934
Mouth Model Accuracy: 99.33775067329407 %
19/19 [=====] - 1s 42ms/step

Confusion Matrix (Mouth Model)
[[300   0]
 [   4 300]]

Classification Report (Mouth Model)

```

	precision	recall	f1-score	support
0	0.99	1.00	0.99	300
1	1.00	0.99	0.99	304
accuracy			0.99	604
macro avg	0.99	0.99	0.99	604
weighted avg	0.99	0.99	0.99	604

APPENDIX – III

PUBLICATION

The project titled “**AI-POWERED DRIVER SAFETY SYSTEM FOR FATIGUE AND DISTRACTION MONITORING**” has been selected for presentation at the **ICCII 2026: 7th International Conference on Computational Intelligence and Industry 5.0**. The conference is organized by *Velammal Institute of Technology, Chennai–Kolkata Highway, Panchetti, Ponneri Taluk, Thiruvallur, Tamil Nadu*. The conference is scheduled to be conducted online on 27th and 28th March 2026. As part of this event, the project will be presented to researchers, academicians, and industry experts, and the corresponding research paper has been submitted for publication in the conference proceedings. This participation provides an opportunity to validate the proposed system, receive valuable expert feedback, and contribute to ongoing research in the fields of artificial intelligence, computer vision, and intelligent transportation systems. It also enhances the practical relevance and academic significance of the project by aligning it with emerging trends in driver safety and smart monitoring technologies.



VELAMMAL
INSTITUTE OF TECHNOLOGY



**7th INTERNATIONAL CONFERENCE ON
COMPUTATIONAL INTELLIGENCE AND INDUSTRY 5.0**

ICCII - 2026

27th & 28th March 2026 | Tamilnadu, India

CERTIFICATE OF PARTICIPATION

This certificate is proudly presented to

MR R. KANNAN

SARANATHAN COLLEGE OF ENGINEERING

has presented a paper titled

**AI-POWERED DRIVER SAFETY SYSTEM FOR FATIGUE AND DISTRACTION
MONITORING**

in the 7th International Conference on Computational Intelligence and Industry 5.0 - ICCII 2026 (ISBN Number: 978-81-985448-0-3) jointly organised by Dept. of Mechanical Engineering, Mechatronics Engineering & Electrical and Electronics Engineering on 27th and 28th March 2026 at Velammal Institute of Technology, Chennai, India.

Dr. M. S. Heaven Dani
Program Chair
Head - Mechanical
VelammaliTech, Chennai

Dr. D. Magesh Babu
Program Chair
Head - Mechatronics
VelammaliTech, Chennai

Dr. S. Muthukaruppasamy
Program Chair
Head - EEE
VelammaliTech, Chennai

Dr. S. Soundararajan
Conference Co-Chair
Vice Principal
VelammaliTech, Chennai

Dr. N. Balaji
Conference Chair
Principal
VelammaliTech, Chennai



VELAMMAL
INSTITUTE OF TECHNOLOGY



**7th INTERNATIONAL CONFERENCE ON
COMPUTATIONAL INTELLIGENCE AND INDUSTRY 5.0**

ICCII - 2026

27th & 28th March 2026 | Tamilnadu, India

CERTIFICATE OF PARTICIPATION

This certificate is proudly presented to

ABISHEK S

SARANATHAN COLLEGE OF ENGINEERING

has presented a paper titled

**AI-POWERED DRIVER SAFETY SYSTEM FOR FATIGUE AND DISTRACTION
MONITORING**

in the 7th International Conference on Computational Intelligence and Industry 5.0 - ICCII 2026 (ISBN Number: 978-81-985448-0-3) jointly organised by Dept. of Mechanical Engineering, Mechatronics Engineering & Electrical and Electronics Engineering on 27th and 28th March 2026 at Velammal Institute of Technology, Chennai, India.

Dr. M. S. Heaven Dani
Program Chair
Head - Mechanical
VelammaliTech, Chennai

Dr. D. Magesh Babu
Program Chair
Head - Mechatronics
VelammaliTech, Chennai

Dr. S. Muthukaruppasamy
Program Chair
Head - EEE
VelammaliTech, Chennai

Dr. S. Soundararajan
Conference Co-Chair
Vice Principal
VelammaliTech, Chennai

Dr. N. Balaji
Conference Chair
Principal
VelammaliTech, Chennai



VELAMMAL
INSTITUTE OF TECHNOLOGY



**7th INTERNATIONAL CONFERENCE ON
COMPUTATIONAL INTELLIGENCE AND INDUSTRY 5.0**

ICCII - 2026

27th & 28th March 2026 | Tamilnadu, India

CERTIFICATE OF PARTICIPATION

This certificate is proudly presented to

AKSHAYAGIRI K

SARANATHAN COLLEGE OF ENGINEERING

has presented a paper titled

**AI-POWERED DRIVER SAFETY SYSTEM FOR FATIGUE AND DISTRACTION
MONITORING**

in the 7th International Conference on Computational Intelligence and Industry 5.0 - ICCII 2026 (ISBN Number: 978-81-985448-0-3) jointly organised by Dept. of Mechanical Engineering, Mechatronics Engineering & Electrical and Electronics Engineering on 27th and 28th March 2026 at Velammal Institute of Technology, Chennai, India.

Dr. M. S. Heaven Dani
Program Chair
Head - Mechanical
VelammaliTech, Chennai

Dr. D. Magesh Babu
Program Chair
Head - Mechatronics
VelammaliTech, Chennai

Dr. S. Muthukaruppasamy
Program Chair
Head - EEE
VelammaliTech, Chennai

Dr. S. Soundararajan
Conference Co-Chair
Vice Principal
VelammaliTech, Chennai

Dr. N. Balaji
Conference Chair
Principal
VelammaliTech, Chennai



VELAMMAL
INSTITUTE OF TECHNOLOGY



**7th INTERNATIONAL CONFERENCE ON
COMPUTATIONAL INTELLIGENCE AND INDUSTRY 5.0**

ICCII - 2026

27th & 28th March 2026 | Tamilnadu, India

CERTIFICATE OF PARTICIPATION

This certificate is proudly presented to

KARTHIK P

SARANATHAN COLLEGE OF ENGINEERING

has presented a paper titled

**AI-POWERED DRIVER SAFETY SYSTEM FOR FATIGUE AND DISTRACTION
MONITORING**

in the 7th International Conference on Computational Intelligence and Industry 5.0 - ICCII 2026 (ISBN Number: 978-81-985448-0-3) jointly organised by Dept. of Mechanical Engineering, Mechatronics Engineering & Electrical and Electronics Engineering on 27th and 28th March 2026 at Velammal Institute of Technology, Chennai, India.

Dr. M. S. Heaven Dani
Program Chair
Head - Mechanical
VelammaliTech, Chennai

Dr. D. Magesh Babu
Program Chair
Head - Mechatronics
VelammaliTech, Chennai

Dr. S. Muthukaruppasamy
Program Chair
Head - EEE
VelammaliTech, Chennai

Dr. S. Soundararajan
Conference Co-Chair
Vice Principal
VelammaliTech, Chennai

Dr. N. Balaji
Conference Chair
Principal
VelammaliTech, Chennai

REFERENCES

- [1] J. Vicente, P. Laguna, A. Bartra, and R. Bailón, “Driver fatigue detection based on heart rate variability,” *IEEE Transactions on Intelligent Transportation Systems*, 2016.
- [2] **L. M. Bergasa, J. Nuevo, and M. A. Sotelo, “Real-time driver fatigue detection based on steering wheel movements,” *IEEE Intelligent Transportation Systems Conference*, 2006.**
- [3] **O. Wathiq and B. D. Ambudkar, “Optimized driver safety through driver fatigue detection methods,” *International Conference on Trends in Electronics and Informatics (ICEI)*, 2017.**
- [4] M. Khan, S. Rahman, and A. Ahmed, “Driver drowsiness detection using vehicle dynamics and machine learning,” *International Journal of Intelligent Transportation Systems*, 2021.
- [5] R. Sharma and P. Verma, “Real-time driver drowsiness detection using computer vision and deep learning,” *International Journal of Advanced Computer Science and Applications*, 2022.
- [6] L. Zhang, Y. Chen, and H. Li, “Driver drowsiness detection using steering behavior and lane deviation analysis,” *Journal of Intelligent Transportation Systems*, 2022.
- [7] **T.-H. Lin, J.-R. Hsieh, and W.-H. Wang, “EEG-based driver drowsiness detection using wavelet transform,” *IEEE International Conference on Systems, Man and Cybernetics*, 2005.**
- [8] M. J. Flores, J. M. Armingol, and A. de la Escalera, “Driver drowsiness warning system using visual information for both diurnal and nocturnal illumination conditions,” *EURASIP Journal on Advances in Signal Processing*, 2010.
- [9] T. Pilutti and A. G. Ulsoy, “Detection of driver drowsiness using steering wheel angle,” *Journal of Dynamic Systems, Measurement, and Control*, 1999.
- [10] **S. Singh and N. Papanikolopoulos, “Monitoring driver fatigue using facial analysis techniques,” *IEEE Intelligent Transportation Systems Conference*, 1999.**
- [11] Q. Ji and X. Yang, “Real-time eye, gaze, and face pose tracking for monitoring driver vigilance,” *IEEE Transactions on Systems, Man, and Cybernetics*, 2002.
- [12] Tawari and M. M. Trivedi, “Robust and continuous estimation of driver gaze zone by dynamic analysis of multiple face videos,” *IEEE Transactions on Intelligent Transportation Systems*, 2014.

- [13] **R. Ghoddosian, M. Galib, and V. Athitsos, “A realistic dataset and baseline temporal model for early drowsiness detection,” *IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2019.**
- [14] **Y. Zhang, G. Pan, K. Jia, M. Lu, Y. Wang, and Z. Wu, “Driver fatigue detection based on eye state recognition,” *IEEE International Conference on Intelligent Transportation Systems*, 2017.**
- [15] **H. Abtahi, B. Hariri, and S. Shirmohammadi, “Driver drowsiness monitoring based on yawning detection,” *IEEE International Instrumentation and Measurement Technology Conference*, 2011.**
- [16] **A. Kumar and R. Patra, “Real-time driver drowsiness detection using deep learning and computer vision,” *IEEE Access*, 2021.**
- [17] **S. Park, F. Pan, S. Kang, and C. D. Yoo, “Driver drowsiness detection system based on feature representation learning using various deep networks,” *IEEE International Conference on Image Processing*, 2016.**
- [18] **R. Sharma and P. Verma, “Real-time driver drowsiness detection using deep learning and computer vision techniques,” *International Journal of Advanced Computer Science and Applications*, 2022.**
- [19] **M. Ali, A. Hussain, and M. Khan, “Driver drowsiness detection using convolutional neural networks and facial features,” *IEEE International Conference on Artificial Intelligence and Signal Processing*, 2020.**